PROJECT REPORT

CLOUD ARCHITECTURE DESIGN – AWS CLOUD PROJECT

PROJECT TITLE

Kubernetes Cost Optimization Framework for SaaS-Based Educational Platforms using FinOps Intelligence

Submitted by:	K. DARSHAN (24BIT0119) DHANUSHKODI Y (24BIT0143) T. THARUN (24BIT0155) S. SANJAY (24BIT0315)
Programme / Branch / Year:	B.Tech. Information Technology — III Year
Course Code and Name:	BCSE355L – Cloud Architecture Design
Institution:	Vellore Institute of Technology, Vellore
Course Instructor / Guide:	Dr. Priya V
Team Number:	Team Project (4 Students)
GitHub Repository:	https://github.com/Dev-withDarshan/KubeCostFinOps_Cloud_Project_2026
Date of Submission:	21 October 2026

Abstract

Educational software-as-a-service (SaaS) platforms—such as learning management systems, online evaluation portals, and virtual assessment services—exhibit extreme, calendar-governed traffic fluctuations. Concurrent student requests surge by one to two orders of magnitude during scheduled examinations and assignment deadlines, while cluster capacity remains largely idle during off-peak and recess periods. Default Kubernetes cluster configurations rely on static, conservative resource allocations to avoid out-of-memory crashes or CPU throttling, leading to pervasive cluster over-provisioning and substantial cloud financial waste. This project develops and evaluates an end-to-end FinOps-informed Kubernetes cost optimization framework tailored for educational workloads. The framework integrates live container telemetry ingestion from the Kubernetes Metrics Server and Kube-State-Metrics, a multi-resource cost analyzer modeled on AWS US-East-1 reference pricing ($0.031611/vCPU-hr, $0.004237/GiB-hr), a domain-specific demand forecasting pipeline incorporating diurnal and academic calendar event indicators, and a conservative recommendation engine that enforces high-availability replica retention and strict safety floors (50m CPU, 64Mi RAM). The architecture is designed around enterprise AWS services, featuring Amazon EKS, EC2 On-Demand/Spot node groups, DynamoDB, RDS PostgreSQL, S3, and event-driven Lambda functions, secured via IAM least privilege and KMS envelope encryption. In local Kind cluster benchmarking across 683 HTTP requests with zero errors, configured baseline requests (250m CPU, 256Mi RAM per pod) were safely rightsized to production floors, demonstrating an empirical peak utilization of 15.16 millicores CPU and 6.45 MiB RAM, projecting a modeled monthly cost reduction from $13.08 to $2.69 (79.41% unrounded savings). Completed local evaluations are formally distinguished from target AWS production deployments.

Keywords

Kubernetes; FinOps Intelligence; Cloud Cost Optimization; Workload Rightsizing; Demand Forecasting; Educational SaaS Architecture; AWS EKS.

Table of Contents

Chapter / Section	Core Topics Covered	Page
1. Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Motivation & Significance; 1.4 Scope; 1.5 Organisation	**4**
2. Literature Survey / Related Work	2.1 Survey of 15 Papers (Table 0); 2.2 Synthesis; 2.3 Individual Research Gaps (Table 1)	**6**
3. Research Gap Analysis	3.1 Domain-Specific Gaps; 3.2 Methodological Deficiencies in Existing Autoscaling	**18**
4. Project Objectives	4.1 SMART Objectives (1-5); 4.2 Expected Deliverables and Assessment Criteria	**19**
5. Novelty Summary	5.1 Central Novelty Statement; 5.2 Novelty Dimension Comparison Matrix (Table 2)	**21**
6. Proposed System and Architecture	6.1 Overview; 6.2 AWS Architecture (Fig 1); 6.3 System Architecture (Fig 2); 6.4 Workflow (Fig 3); 6.5 Security	**22**
7. Dataset Description	7.1 Academic Workload Telemetry (Table 3); 7.2 Data Preparation & Leakage Control	**28**
8. AWS Services Planning	8.1 AWS Services Matrix (Table 4); 8.2 Deployment Design; 8.3 Cost Estimation Breakdown	**29**
9. Methodology and Implementation Plan	9.1 Overall Methodology; 9.2 Tools (Table 5); 9.3 Implementation Steps; 9.4 Evaluation Metrics	**33**
10. Implementation / Development	10.1 Implemented Modules (Table 6); 10.2 Configuration Evidence; 10.3 GitHub Repository	**36**
11. Testing, Results and Observations	11.1 Test Plan (Table 7); 11.2 Controlled Benchmark (Figs 4, 5, 6); 11.3 Adverse Trial; 11.4 Results Summary	**41**
12. Discussion, Limitations and Risk Analysis	12.1 Findings Discussion; 12.2 Limitations; 12.3 Risks and Mitigation (Table 8)	**48**
13. Team Contribution and GitHub Management	13.1 Team Contribution Matrix (Table 9); 13.2 Git Branch Workflow; 13.3 Contribution Evidence	**50**
14. Conclusion and Future Enhancements	14.1 Concluding Verdict; 14.2 Planned AWS Production Enhancements	**52**
References	Complete APA Citations for Research Papers (P1-P15), Cloud Documentation & Tooling	**53**
Appendix A – Project Timeline	Phase-Wise Execution Milestone Tracking (Table 10)	**57**
Appendix B – Review Checklist	Instructor Review Checklist Compliance Verification	**57**
Appendix C – Reproducibility Notes	CLI Reproduction Commands & Artifact Inventory	**58**

1. Introduction

1.1 Background

Modern higher educational institutions and commercial educational technology (EdTech) providers rely increasingly on software-as-a-service (SaaS) architectures to deliver mission-critical learning management systems (LMS), online examination portals, coding sandbox environments, and distributed digital classrooms. To guarantee elasticity and service isolation, these platforms are deployed predominantly as containerized microservices orchestrated by Kubernetes (Burns et al., 2016). In Kubernetes, container resource allocation is governed by declarative 'requests' (which dictate scheduler reservation and node placement) and 'limits' (which cap kernel cgroup runtime consumption to prevent noisy-neighbor starvation). While these primitives provide robust operational stability, they fail to bridge the divide between infrastructure reservations and financial economics. A container resource request guarantees compute headroom on an underlying virtual host, incurring cloud leasing costs regardless of whether the allocated CPU cycles and memory pages are actively utilized. Consequently, cloud financial operations (FinOps) has emerged as an essential engineering discipline uniting technology, finance, and operations to establish continuous cost visibility, accountable attribution, and automated workload rightsizing (FinOps Foundation, 2024).

1.2 Problem Statement

Educational SaaS platforms operate under an acute operational contradiction: extreme, time-delimited demand volatility coupled with a zero-tolerance threshold for service disruption. Workload arrivals are tightly synchronized with the academic calendar; normal instructional hours produce moderate, predictable API traffic, whereas scheduled mid-semester examinations, finals, and assignment submission deadlines trigger demand spikes that rise by one to two orders of magnitude within minutes. To prevent catastrophic container out-of-memory (OOM) terminations or CPU-throttled request timeouts during exams, DevOps engineers systematically configure resource requests at peak-concurrency ceilings. Because these peak events occupy less than 5% of the operational academic calendar, educational clusters remain severely over-provisioned for the remaining 95% of the year. Industry studies and empirical research indicate that average enterprise container utilization rarely exceeds 10–15% of configured requests, resulting in substantial financial waste. Conversely, naive reactive autoscaling (such as standard

Kubernetes HPA) exhibits latency lag during sudden examination bursts, risking severe student disruption. Therefore, educational institutions require a domain-aware FinOps framework that anticipates calendar events, models AWS cloud costs, and executes conservative rightsizing with strict safety floors.

1.3 Motivation and Significance

Public and private educational institutions operate under stringent annual operational budgets, where cloud infrastructure overspending directly detracts from academic research, faculty hiring, and student scholarship funds. Commercial Kubernetes cost monitoring platforms (such as proprietary enterprise offerings) are often prohibitively expensive to license and operate as opaque black boxes that hide their underlying pricing equations. Furthermore, generic cloud autoscalers possess zero contextual awareness of academic calendar schedules, treating an examination morning identically to an ordinary weekend recess. Developing an open, transparent, and auditable FinOps framework grounded in Kubernetes telemetry and domain forecasting enables institutions to achieve substantial cloud savings (typically 20% to 40% across production fleets) while maintaining deterministic performance guarantees and high-availability safety guardrails during critical assessments.

1.4 Scope and Operational Boundaries

To maintain academic and scientific integrity, this project establishes a clear, auditable distinction between the empirical work completed within the local Kind testbed prototype and the target architectural design planned for enterprise AWS production deployment.

Architectural Dimension	Completed Local Evaluation Scope (Evidenced)	Planned AWS Production Scope (Architected)
Container Orchestration	Local Kind cluster (`kubecost-finops-local`, K8s v1.37.0) on Docker Desktop with 2 workload replicas (`edusaas-backend`).	Multi-AZ Amazon EKS cluster with managed On-Demand and Spot EC2 worker node groups across private subnets.
Telemetry Ingestion	Kubernetes Metrics Server v0.7.1 and Kube-State-Metrics v2.12.0 scraping live Linux cgroups and object states.	Amazon Managed Service for Prometheus pulling container metrics integrated with OpenCost Helm charts.
FinOps Engine & Pricing	Python standard library multi-resource cost analyzer applying AWS US-East-1 reference pricing ($0.031611/vCPU-hr, $0.004237/GiB-hr).	Real-time AWS Cost Explorer API integration, Cost and Usage Reports (CUR), and multi-tenant department showback.

Architectural Dimension	Completed Local Evaluation Scope (Evidenced)	Planned AWS Production Scope (Architected)
Workload Forecasting	Synthetic educational time-series generator (`ml_forecast.py`) outputting diurnal and exam features across 6 domain fields.	Production Amazon SageMaker endpoint hosting distributed Prophet/LSTM forecasters retrained on historical LMS data.
Rightsizing Action Mode	Non-mutating dry-run rightsizing recommendations with +20% safety buffer and 50m/64Mi safety floors.	Automated GitOps / ArgoCD deployment reconciliation and event-driven AWS Lambda rightsizing with approval gates.
Verification & Testing	54 passing regression unit tests, 10 experiment forensic checks, and 28 release readiness verifications.	Enterprise chaos engineering (AWS Fault Injection Simulator) and multi-region disaster recovery failover.

1.5 Report Organisation

This report is structured into 14 comprehensive chapters following the instructor's submission requirements: Chapter 2 presents an in-depth literature survey of 15 peer-reviewed papers (P1–P15), comparative synthesis, and individual research gaps. Chapter 3 consolidates the core research gaps. Chapter 4 establishes specific project objectives and expected outcomes. Chapter 5 details the novelty claim and multi-dimensional comparison. Chapter 6 illustrates the proposed AWS and Complete System Architectures, complete with workflow diagrams and security specifications. Chapter 7 describes the workload dataset. Chapter 8 provides detailed AWS service planning and cost calculations. Chapter 9 outlines the methodology, tools, and evaluation metrics. Chapter 10 documents the implemented software modules, configuration evidence, and GitHub repository structure. Chapter 11 details empirical test cases, the authoritative controlled benchmark run, adverse counter-evidence, and results summary. Chapter 12 provides a critical discussion of limitations and risks. Chapter 13 presents the team contribution matrix and Git workflow. Chapter 14 concludes the report and identifies future AWS production enhancements, followed by APA References and Appendices A, B, and C.

2. Literature Survey / Related Work

A rigorous, systematic literature review was conducted covering 15 peer-reviewed research publications published predominantly between 2023 and 2026 across leading venues (IEEE Transactions, ACM SIGCOMM/KDD, Springer, Elsevier, USENIX OSDI, and Scopus-indexed journals), supplemented by foundational systems papers establishing core Kubernetes and cloud

time-series principles. Table 0 summarizes the 15 surveyed papers, documenting their full citations, problems addressed, methodology, evaluation datasets, and key findings.

Table 0. Literature Survey Summary (15 Research Papers Evaluated).

ID	Full Citation & DOI	Problem Addressed	Method / Architecture	Dataset / Evaluation	Key Findings
P1	Ahmad, H., Treude, C., Wagner, M., & Szabo, C. (2025). "Towards resource-efficient reactive and proactive auto-scaling for microservice architectures". Journal of Systems and Software, 225, Article 112390. DOI: https://doi.org/10.1016/j.jss.2025.112390	Delays, oscillation, and simultaneous over- and under-provisioning in default Kubernetes Horizontal Pod Autoscaling (HPA) when handling bursty microservice workloads.	Smart HPA (reactive heuristic) and ProSmart HPA (proactive ML-driven autoscaling) that cooperatively share node capacity across interdependent microservices.	Sock Shop and TrainTicket microservice benchmarks under fluctuating synthetic and trace-driven arrival rates.	Reduced over-provisioning by up to 34% and under-provisioning by 28% compared to standard Kubernetes HPA without violating p95 latency thresholds.
P2	Pothu, S. N., & Kailasam, S. (2024). "Effective priority-based resource allocation for proactive auto-scaling framework in workload prediction using hybrid tree-enhanced vector machine model". Discover Sustainability, 5, Article 391. DOI: https://doi.org/10.1007/s43621-024-00583-x	High operational energy waste and resource misallocation in cloud data centers resulting from unpredictable demand swings.	Hybrid Tree-Enhanced Support Vector Machine (HTE-SVM) combined with priority-based proactive resource allocation across virtual machines.	Real-world web workload traces (NASA and ClarkNet server access logs) evaluated for prediction accuracy and energy expenditure.	Achieved 94.2% workload forecasting accuracy and reduced cloud data center operational energy consumption by 18.5%.
P3	Tran, M.-N., & Kim, Y. (2024). "Optimized resource usage with hybrid auto-scaling system for Knative serverless edge computing". Future Generation Computer Systems, 152, 304–316. DOI: https://doi.org/10.1016/j.future.2023.11.010	Cold-start latency spikes and memory over-allocation in serverless container platforms (Knative) hosted on edge-cloud nodes.	Hybrid autoscaling architecture combining proactive ARIMA traffic forecasting with fine-grained vertical container rightsizing.	Synthetic and real trace-driven web request streams evaluated on a physical multi-node Kubernetes/Knative testbed.	Decreased tail request latency by 22% while cutting memory footprint by 31.4% across distributed edge nodes.

ID	Full Citation & DOI	Problem Addressed	Method / Architecture	Dataset / Evaluation	Key Findings
P4	Zhang, H., Guo, T., Tian, W., & Ma, H. (2024). "Learning-driven hybrid scaling for multi-type services in cloud". Journal of Parallel and Distributed Computing, 189, Article 104880. DOI: https://doi.org/10.1016/j.jpdc.2024.104880	Balancing strict Service Level Objective (SLO) compliance against cloud infrastructure leasing costs across heterogeneous multi-tier services.	Ensemble deep learning predictor coupled with reinforcement learning for concurrent horizontal and vertical scaling with anomaly compensation.	Alibaba Cluster Trace 2018 and Google Cluster Trace 2019 evaluated for response time violations and virtual instance hourly costs.	Maintained >99.2% SLO compliance while decreasing overall virtual compute leasing expenses by 24.6%.
P5	Alidoost Alanagh, Y., Firouzi, M., Rasouli Kenari, A., & Shamsi, M. (2023). "Introducing an adaptive model for auto-scaling cloud computing based on workload classification". Concurrency and Computation: Practice and Experience, 35(22), Article e7720. DOI: https://doi.org/10.1002/cpe.7720	Standard autoscalers applying a single uniform forecasting algorithm across diverse, non-stationary workload patterns.	Unsupervised workload classification (clustering traces into bursty, periodic, and steady regimes) to dynamically select the best-fit predictor.	Real-world web server access logs (University of Saskatchewan and Calgary web server traces).	Improved autoscaling provisioning responsiveness by 29% and reduced prediction error by 33% across regime transitions.
P6	Saravanan, G., & Santhosh Babu, A. V. (2024). "Workload prediction for enhancing power efficiency of cloud data centers using optimized self-attention-based progressive generative adversarial network". International Journal of Communication Systems, 37(1), Article e5634. DOI: https://doi.org/10.1002/dac.5634	Inability of conventional shallow neural networks to capture long-range temporal dependencies and extreme burst peaks in enterprise cloud workloads.	Self-Attention Progressive Generative Adversarial Network (SA-PGAN) optimizing multi-step workload time-series generation.	PlanetLab and Google Cluster traces evaluating Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and server power consumption.	Reduced workload forecasting RMSE to 0.042 and delivered an estimated 16.8% reduction in cloud server power overhead.

ID	Full Citation & DOI	Problem Addressed	Method / Architecture	Dataset / Evaluation	Key Findings
P7	Jia, K., Xiang, J., & Li, B. (2024). "DuCFF: A dual-channel feature-fusion network for workload prediction in a cloud infrastructure". Electronics, 13(18), Article 3588. DOI: https://doi.org/10.3390/electronics13183588	Feature decoupling where sequential temporal telemetry and exogenous environmental features are processed independently without cross-feature fusion.	Dual-Channel Feature-Fusion Network (DuCFF) combining 1D Convolutional Neural Networks (CNN) with Bidirectional LSTM channels.	Public multi-tenant cloud workload traces incorporating external calendar indicators evaluated on CPU and memory utilization sequences.	Outperformed standard LSTM and GRU baselines with a 19.3% reduction in MAE and 21.5% improvement in peak traffic forecast accuracy.
P8	Alharthi, S., Alshamsi, A., Alseiari, A., & Alwarafy, A. (2024). "Auto-scaling techniques in cloud computing: Issues and research directions". Sensors, 24(17), Article 5551. DOI: https://doi.org/10.3390/sensors24175551	Lack of comprehensive taxonomic standardization and empirical trade-off analysis across modern cloud autoscaling paradigms.	Systematic literature survey analyzing over 120 autoscaling publications spanning reactive, proactive, hybrid, and machine learning techniques.	Taxonomic classification and comparative matrix across AWS, GCP, Azure, and open-source container orchestrators.	Identified critical industry deficiencies: less than 15% of surveyed systems incorporate cost metrics, and under 5% account for domain calendar events.
P9	Xue, S., Qu, C., Shi, X., Liao, C., Zhu, S., Tan, X., Ma, L., Wang, S., Wang, S., Hu, Y., Lei, L., Zheng, Y., Li, J., & Zhang, J. (2022). "A meta reinforcement learning approach for predictive autoscaling in the cloud". Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 4290–4299. DOI: https://doi.org/10.1145/3534678.3539063	Prohibitive cold-start training times and catastrophic forgetting when standard reinforcement learning models encounter unseen cloud workload regimes.	Meta-Reinforcement Learning (Meta-RL) architecture utilizing few-shot policy generalization across diverse multi-tenant cloud workloads.	Industrial production workload traces from Ant Group spanning thousands of online microservices.	Accelerated policy adaptation to new workload profiles by 80% while saving 14.7% in compute allocation without compromising SLOs.

ID	Full Citation & DOI	Problem Addressed	Method / Architecture	Dataset / Evaluation	Key Findings
P10	Qiu, H., Banerjee, S. S., Jha, S., Kalbarczyk, Z. T., & Iyer, R. K. (2020). "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices". Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20), 805–825. DOI: https://www.usenix.org/conference/osdi20/presentation/qiu	Microservice tail-latency degradation and cascading resource contention caused by uncoordinated per-pod autoscaling across deep call graphs.	FIRM framework utilizing adaptive reinforcement learning to identify bottlenecked microservices, mitigate contention, and dynamically rightsize resource limits.	DeathStarBench microservice benchmark suites (Social Network and Media Service) evaluated on a physical 40-node server cluster.	Reduced tail (p99) latency by up to 16x and prevented 90% of SLO violations while reclaiming up to 62% in overallocated CPU cores.
P11	Rzadca, K., Findeisen, P., Świderski, J., Zych, P., Broniek, P., Kusmierek, J., Nowak, P., Strack, B., Witusowski, P., Hand, S., & Wilkes, J. (2020). "Autopilot: Workload autoscaling at Google". Proceedings of the 15th European Conference on Computer Systems (EuroSys '20), Article 16. DOI: https://doi.org/10.1145/3342195.3387524	Vast cluster-wide resource slack caused by human operators conservatively over-sizing resource requests to prevent out-of-memory or throttling crashes.	Google Autopilot combining historical utilization windowing, exponential moving averages, and automated horizontal/vertical sizing algorithms.	Google global production fleet consisting of hundreds of thousands of jobs and millions of containers evaluated over multiple months.	Reduced aggregate cluster resource slack by over 50% across Google's worldwide production container fleet with negligible task disruption.
P12	Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). "Borg, Omega, and Kubernetes". ACM Queue, 14(1), 70–93. DOI: https://doi.org/10.1145/2898442.2898444	Designing robust, decoupled, and declarative container orchestration platforms capable of scaling distributed applications across shared server pools.	Foundational architectural analysis tracing the evolution from Google Borg and Omega to the open-source Kubernetes container management system.	Architectural case study based on more than a decade of Google container orchestration production operations.	Established core cloud-native abstractions: declarative state reconciliation, pod encapsulation, label-based loose coupling, and cgroup isolation.

ID	Full Citation & DOI	Problem Addressed	Method / Architecture	Dataset / Evaluation	Key Findings
P13	Taylor, S. J., & Letham, B. (2018). "Forecasting at scale". The American Statistician, 72(1), 37–45. DOI: https://doi.org/10.1080/00031305.2017.1380080	Difficulty of producing accurate, automated, and robust time-series forecasts across thousands of business metrics with non-linear trends and holidays.	Decomposable additive time-series model (Prophet) combining non-linear trend curves, multi-period Fourier seasonality, and custom holiday/calendar effects.	Thousands of daily and hourly business event and web traffic time-series at Meta (Facebook).	Delivered robust, automated forecasts resilient to missing data, outliers, and structural trend shifts, with human-interpretable parameters.
P14	Mao, H., Schwarzkopf, M., Venkatakrishnan, S. B., Meng, Z., & Alizadeh, M. (2019). "Learning scheduling algorithms for data processing clusters". Proceedings of the ACM SIGCOMM 2019 Conference, 270–288. DOI: https://doi.org/10.1145/3341302.3342080	Hand-tuned heuristic cluster schedulers (FIFO, Fair, DRF) failing to optimize execution times across complex directed acyclic graph (DAG) job workflows.	Decima system employing Graph Neural Networks (GNN) and reinforcement learning to learn workload-aware scheduling policies directly from job DAG structures.	Large-scale Spark and data analytics workload traces evaluated on physical testbed clusters and high-fidelity simulators.	Reduced average job completion time by 21% to 3x compared to state-of-the-art hand-crafted cluster scheduling heuristics.
P15	Mao, H., Alizadeh, M., Menache, I., & Kandula, S. (2016). "Resource management with deep reinforcement learning". Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets '16), 50–56. DOI: https://doi.org/10.1145/3005745.3005750	Multi-resource packing in cluster computing where heuristic bin-packing algorithms make sub-optimal trade-offs between CPU, memory, and time.	DeepRM system translating cluster allocation states directly to scheduling decisions using model-free deep policy gradient reinforcement learning.	Simulated multi-resource computing clusters under synthetic and trace-derived job arrival sequences.	Demonstrated that neural agents can learn effective multi-resource bin-packing and fair-sharing policies purely through environmental reward signals.

2.1 Synthesis and Comparison

Cross-analysis of the surveyed literature reveals distinct research methodologies, architectural assumptions, and trade-offs. The papers can be synthesized into four primary technological categories:

1. Reactive vs. Proactive Autoscaling (P1, P2, P3, P8): Standard reactive mechanisms (such as default Kubernetes HPA) scale solely in response to threshold breaches. As established by Ahmad et al. (P1) and Tran & Kim (P3), reactive scaling introduces unavoidable cold-start latency and frequently triggers over-compensating oscillations. Proactive mechanisms mitigate these delays by projecting near-term traffic. However, existing proactive schedulers rely almost exclusively on past hardware metrics rather than external business or institutional context.

2. Machine Learning and Workload Forecasting (P4, P5, P6, P7, P13): Advanced forecasters employ diverse algorithms, ranging from additive regression (Prophet in P13) to hybrid SVMs (P2), dual-channel CNN-BiLSTMs (P7), and generative adversarial networks (P6). While deep learning models achieve impressive error reductions on benchmark traces (e.g., PlanetLab, Alibaba), they introduce prohibitive training overheads and require massive historical datasets that educational institutions rarely possess. Conversely, lightweight statistical models (P13) provide interpretable, event-conditioned extrapolations suitable for edge deployment.

3. Reinforcement Learning for Cluster Scheduling (P9, P10, P14, P15): Seminal works by Mao et al. (P14, P15) and Xue et al. (P9) demonstrate that neural agents can learn complex job packing and multi-resource allocation directly from experience. Furthermore, FIRM (P10) highlights the importance of microservice SLO-preservation. However, online RL poses significant operational risks: exploration phases can cause sudden service crashes during high-stakes examinations, making unconstrained RL unacceptable in production educational environments without hard safety floors.

4. Container Orchestration & Systems Realities (P11, P12): Google Autopilot (P11) and Borg/Kubernetes systems papers (P12) provide the empirical bedrock of modern container rightsizing. Autopilot proves that multi-window utilization tracking can safely reclaim over 50%

of cluster slack. However, as Alharthi et al. (P8) observe in their comprehensive survey, less than 15% of existing autoscalers integrate explicit cloud monetary cost models, and fewer than 5% incorporate academic calendar features.

2.2 Individual Research Gap Analysis

In accordance with course guidelines, each student team member independently analyzed their assigned research papers to evaluate existing methods, identify technical advantages and limitations, formulate research gaps in their own words, and propose actionable improvements. Table 1 presents this comprehensive individual analysis across all four students.

Table 1. Individual Research Gap Analysis and Proposed Improvements.

Paper & Student	Existing Method	Advantages	Limitations	Research Gap Identified by Student	Possible Improvement
P1 (Ahmad) [K. Darshan]	Proactive and reactive HPA algorithms sharing cluster resources across microservices based purely on CPU/memory utilization thresholds.	Demonstrates that reactive and proactive policies can be coupled effectively; provides rigorous evaluation of over- and under-provisioning trade-offs.	Focuses strictly on raw hardware allocation; does not model financial cost metrics, cloud billing structures, or domain-specific calendar demand surges.	Evaluates resource efficiency but lacks dollar-based cost attribution and does not incorporate domain calendar events (e.g., examination schedules) into scaling decisions.	Combine academic calendar features with container resource telemetry, mapping rightsizing recommendations directly to AWS reference hourly pricing.
P2 (Pothu) [T. Tharun]	Hybrid Decision Tree and Support Vector Machine (HTE-SVM) model classifying workload demand into operational priority tiers.	High prediction accuracy on historical web traces; establishes a priority-tiered scheduling hierarchy for non-critical workloads.	Evaluated at the coarse virtual machine (VM) level rather than containerized microservices; does not provide Kubernetes pod-level telemetry.	VM-level workload classification does not translate directly to granular container cgroups, pod replica counts, or Kubernetes namespace cost showback.	Map priority-based demand classification directly to Kubernetes pod resource requests and limits, enforcing high-availability replica floors.

Paper & Student	Existing Method	Advantages	Limitations	Research Gap Identified by Student	Possible Improvement
P3 (Tran) [K. Darshan]	ARIMA-based proactive horizontal pod scaling coupled with vertical pod memory resizing in a Knative serverless environment.	Addresses both replica count and vertical sizing simultaneously; provides concrete evaluation on real containerized testbed infrastructure.	Serverless scale-to-zero model is unsuited for stateful educational portals and long-lived database connections; lacks cloud financial cost modeling.	The serverless edge environment assumes scale-to-zero semantics, whereas enterprise educational SaaS requires persistent HA floors and explicit FinOps visibility.	Retain persistent minimum replica floors (e.g., 2 replicas) for high availability and calculate cost savings using transparent cloud hourly billing rates.
P4 (Zhang) [T. Tharun]	Ensemble neural network forecasting coupled with Q-learning reinforcement learning for multi-service autoscaling.	Simultaneously optimizes response time and compute leasing cost; accounts for complex multi-service interaction effects.	High computational complexity and training overhead; complex reward functions make policy decisions opaque and difficult to audit.	Complex multi-service RL policies cannot be safely deployed without transparent safety guardrails and dry-run audit mechanisms.	Implement an interpretable, rule-governed recommendation engine with +20% safety headroom and auditable dry-run validation before applying actions.
P5 (Alidoost Alanagh) [Dhanushkodi Y]	K-means clustering to classify workload segments, routing each segment to specialized regression or time-series forecasters.	Demonstrates that classifying workload regimes before forecasting significantly reduces prediction error compared to a single static model.	Evaluated exclusively on offline historical web server logs; does not integrate live Kubernetes cgroup metrics or FinOps cost visibility.	Offline web log classification does not provide live cluster telemetry ingestion or tie predicted traffic surges to dollar-cost savings.	Ingest live Kubernetes Metrics API telemetry in real time and categorize workload phases (Baseline, Low Load, Exam Surge, Recovery) for tailored FinOps rightsizing.
P6 (Saravanan) [K. Darshan]	Generative Adversarial Network with self-attention layers generating synthetic workload forecasts to preemptively spin up capacity.	Captures complex temporal correlations and sharp workload bursts better than standard recurrent neural networks.	Extremely heavy training and inference computation; requires massive historical training datasets; prone to mode collapse during sudden shifts.	Heavy neural architectures require GPU infrastructure that increases cost, defeating the purpose of lightweight FinOps cost optimization for smaller institutions.	Employ lightweight statistical and additive calendar-aware models (e.g., Prophet-style diurnal and event-based forecasting) deployable on standard CPU instances.

Paper & Student	Existing Method	Advantages	Limitations	Research Gap Identified by Student	Possible Improvement
P7 (Jia) [K. Darshan]	Two-branch neural architecture processing temporal metric sequences in parallel with exogenous context before feature fusion.	Explicitly demonstrates the importance of fusing external domain indicators with internal performance metrics for accurate peak prediction.	Does not close the operational loop: outputs raw predicted numbers without translating them into actionable Kubernetes scaling or rightsizing actions.	Feature fusion stops at numerical prediction and does not generate safe Kubernetes pod resource specifications or assess FinOps economic impact.	Connect exogenous feature fusion (academic calendar exam flags, deadlines) directly to an automated FinOps rightsizing engine with safety floors.
P8 (Alharthi) [T. Tharun]	Taxonomic survey categorizing autoscalers by architecture, trigger metrics, decision mechanisms, and cloud deployment models.	Provides an authoritative, holistic overview of the current state of cloud autoscaling and clearly articulates open research frontiers.	Secondary survey paper without an original empirical software prototype or experimental benchmark.	Confirms the widespread absence of FinOps cost accountability and domain-specific calendar integration in production autoscalers.	Directly address the primary gaps identified by Alharthi et al. by building an end-to-end framework uniting FinOps cost attribution with calendar-aware scaling.
P9 (Xue) [Dhanushkodi Y]	Model-agnostic meta-learning training policy networks across multiple task distributions to achieve fast few-shot online adaptation.	Overcomes the slow convergence of standard RL in dynamic environments; validated on massive enterprise-scale production microservices.	Requires sophisticated distributed training infrastructure and access to massive cross-workload historical repositories.	Meta-RL is over-engineered for educational institutions that operate predictable, calendar-governed academic cycles rather than thousands of arbitrary services.	Use deterministic academic calendar rules and statistical moving averages that provide predictable, explainable rightsizing without training overhead.
P10 (Qiu) [S. Sanjay]	Dynamic reinforcement learning tracking critical latency paths and applying fine-grained vertical container resource adjustments.	Pioneering work in microservice-aware resource isolation and SLO preservation; rigorous physical cluster evaluation.	High systems overhead requiring kernel-level eBPF instrumentation; focuses on low-level contention rather than monthly cloud dollar expenditure.	Focuses on microsecond-level CPU throttling and tail latency rather than cloud cost modeling, educational demand forecasting, or FinOps accountability.	Translate resource rightsizing into transparent AWS reference monthly dollar costs while establishing explicit minimum resource floors (50m/64Mi) to prevent tail latency spikes.

Paper & Student	Existing Method	Advantages	Limitations	Research Gap Identified by Student	Possible Improvement
P11 (Rzadca) [S. Sanjay]	Multi-window historical utilization tracking combining moving averages with decay factors to recommend rightsized task limits and task counts.	The definitive empirical benchmark for production container rightsizing at scale; proves that automated rightsizing can safely reclaim >50% slack.	Proprietary internal Google Borg architecture; tightly coupled to internal telemetry and lacks public cloud billing awareness or calendar hooks.	Autopilot operates in private corporate infrastructure without public cloud multi-tenant billing models (AWS On-Demand/Spot) or academic event conditioning.	Adapt Autopilot's proven moving-window rightsizing principles to standard Kubernetes and open AWS reference pricing with domain calendar awareness.
P12 (Burns) [K. Darshan]	Declarative desired-state control loops continuously reconciling actual cluster state against user-specified YAML manifests.	Defines the fundamental architectural philosophy and container abstractions governing modern Kubernetes cloud engineering.	Foundational systems architecture paper; does not address autonomous cloud financial engineering, FinOps showback, or modern public cloud pricing.	Kubernetes reconciles pod requests declared by humans but lacks internal economic intelligence to verify whether declared requests are financially wasteful.	Build an external FinOps reconciliation loop that inspects live cgroup utilization and proposes rightsized declarative pod manifests.
P13 (Taylor) [Dhanushkodi Y]	Decomposable additive regression model: $y(t) = g(t) + s(t) + h(t) + e(t)$ modeling trend, seasonality, and holiday events.	Excels at incorporating custom calendar events (holidays, exams) with interpretable coefficients; robust to missing data and regime shifts.	Designed for macro business metrics; requires integration scaffolding to ingest high-frequency container telemetry and output Kubernetes scaling actions.	Calendar-aware forecasting has not been integrated into a closed-loop Kubernetes FinOps engine to trigger pre-emptive scaling before exam traffic surges.	Incorporate Prophet's additive calendar formulation into the FinOps forecasting pipeline to model academic examination windows and assignment deadlines.

Paper & Student	Existing Method	Advantages	Limitations	Research Gap Identified by Student	Possible Improvement
P14 (Mao) [K. Darshan]	Graph Neural Networks embedding job DAG dependencies, trained via policy gradient reinforcement learning to schedule tasks.	Demonstrates that learned policies can drastically outperform human-engineered heuristics in complex multi-resource scheduling environments.	Targeted at big-data batch DAG computation (Spark) rather than continuous, interactive educational web application microservices.	Batch job DAG scheduling does not address continuous HTTP request-driven educational SaaS workloads or monthly public cloud cost attribution.	Focus on interactive SaaS request telemetry and establish deterministic rightsizing algorithms that provide immediate, verifiable FinOps savings.
P15 (Mao) [S. Sanjay]	Deep policy gradient neural network observing 2D image-like representations of cluster resource allocation to place tasks.	Foundational proof-of-concept demonstrating that reinforcement learning can solve multi-resource allocation without handcrafted heuristics.	Simulated discrete time-step environment; did not evaluate real Linux cgroups, live network jitter, or commercial cloud billing structures.	Simulated RL allocation does not establish practical, auditable cost savings in live Kubernetes clusters hosting latency-sensitive educational services.	Implement transparent, inspectable FinOps cost formulas grounded in empirical Kubernetes Metrics Server cgroup measurements and AWS reference rates.

2.3 Consolidated Research Gap

Synthesizing the individual gaps across all 15 evaluated papers establishes a singular, consolidated research deficit: Existing cloud and container management research remains bifurcated between low-level reactive scheduling heuristics (which ignore cloud financial billing structures) and academic machine-learning algorithms (which optimize synthetic benchmark traces in simulation without integrating live container telemetry or enforcing high-availability production safety floors). Specifically, no existing open-source framework bridges the operational gap between Kubernetes Linux cgroup metrics, transparent public cloud pricing models (e.g., AWS EC2 On-Demand/Spot hourly rates), and domain-specific calendar exogenous factors (e.g., exam periods, assignment deadlines). This project directly addresses this consolidated gap by building a verifiable, end-to-end FinOps intelligence pipeline that converts real-time cluster telemetry into financially quantified, safety-clamped rightsizing recommendations tailored for educational SaaS platforms.

3. Research Gap Analysis

Building upon the literature survey, this chapter formalizes the four critical technical gaps that hinder effective cloud cost optimization in educational Kubernetes environments:

3.1 Gap 1: Inadequate Handling of Cyclical and Bursty Academic Calendars

Commercial and open-source autoscalers (such as Kubernetes HPA and AWS Auto Scaling) treat incoming traffic as an arbitrary stochastic process. They compute moving averages or threshold percentages over recent 1- to 5-minute metric windows. In an educational platform, traffic surges are not random anomalies; they are deterministic, synchronized events dictated by institutional course schedules, exam timetables, and submission deadlines. Because reactive autoscalers remain unaware of the academic calendar, they begin spinning up additional pod replicas only after latency has already spiked and students experience timeouts. Conversely, heavy deep learning predictors (such as LSTM or GAN networks in P6 and P7) fail to adapt quickly when exam schedules shift, requiring expensive model retraining.

3.2 Gap 2: Disconnection between Container Request Schedulers and Cloud Invoices

Kubernetes schedules workloads based on user-declared 'requests', but cloud providers invoice customers based on provisioned virtual machine instances (e.g., AWS EC2 hourly rates). Traditional Kubernetes dashboards display CPU millicores and memory megabytes, leaving engineering teams blind to the dollar consequences of over-provisioning. Existing commercial tools that offer cost allocation (such as Kubecost) frequently require complex proprietary agents and do not provide an inspectable, deterministic pricing engine that can be audited and verified offline. There is an acute need for a lightweight, transparent FinOps engine that directly translates container cgroup reservations into exact AWS dollar line items using verifiable unit formulas.

3.3 Gap 3: Absence of Deterministic High-Availability Safety Floors in Rightsizing

Existing algorithmic rightsizing solutions (including Google Autopilot in P11 and RL-based scalers in P4 and P10) aim to minimize aggregate resource slack by aggressively scaling down pod requests to match recent average usage. In an educational SaaS application, over-aggressive

downscaling during off-peak periods can reduce pod allocations to extreme minimums (e.g., 5m CPU, 10Mi RAM). When hundreds of students suddenly log in simultaneously to access an online exam, these constricted pods experience immediate CPU throttling, socket starvation, or OOM terminations before horizontal scaling can take effect. Existing frameworks lack mandatory, immutable safety floors that preserve high-availability redundancy regardless of how low recent utilization drops.

3.4 Gap 4: Lack of Lightweight, Non-Mutating, and Auditable FinOps Pipelines

Many advanced autoscaling frameworks proposed in academia operate either in pure software simulators (P15) or execute live cluster mutations without human oversight (P4, P9). In mission-critical higher education IT, un-audited automated mutations are strictly prohibited by governance policies. Current platforms lack a transparent dry-run recommendation architecture where suggested rightsizing configurations can be forensically inspected, modeled for savings, and evaluated against safety constraints in an offline dashboard before applying declarative mutations.

4. Project Objectives

To systematically address the research gaps identified above, this project defines five specific, measurable, achievable, realistic, and time-bound (SMART) objectives:

• Objective 1: Telemetry Ingestion Pipeline:

Design and deploy an automated telemetry ingestion pipeline capable of polling Kubernetes Metrics Server and Kube-State-Metrics APIs at 2-second intervals, extracting container cgroup CPU and memory telemetry, and parsing object metadata without third-party agent overhead. Success Criterion: Achieve 100% data ingestion continuity across 30+ consecutive sampling cycles with zero NaN values or unparsed metric strings.

• Objective 2: FinOps Multi-Resource Cost Modeling Engine:

Develop an auditable FinOps cost analyzer that translates raw container resource requests and limits into hourly and 730-hour monthly dollar costs using published AWS US-East-1 reference pricing ($0.031611/vCPU-hr, $0.004237/GiB-hr) with exact binary GiB conversions. Success

Criterion: Eliminate mathematical rounding drift, ensuring zero discrepancy between component sums and stored totals across all cost allocations.

• Objective 3: Academic Calendar Demand Forecaster:

Implement a lightweight demand forecasting pipeline incorporating diurnal circadian curves, examination period flags (Tuesday/Thursday peaks), assignment submission deadlines (Sunday/Wednesday surges), and active student counts across 6 domain fields. Success Criterion: Expose standard REST forecast endpoints responding within <50 ms and demonstrating elevated resource projections during simulated academic exam windows.

• Objective 4: Recommendation Engine with Production Safety Floors:

Construct a rule-governed rightsizing recommendation engine that calculates optimized pod configurations with a +20% safety headroom above measured peak usage, enforces mandatory high-availability replica preservation (>=2 replicas), and clamps requests to immutable production safety floors (50m CPU, 64Mi RAM). Success Criterion: Ensure recommendations operate in non-mutating dry-run mode, preserving live cluster workload stability.

• Objective 5: Multi-Phase Controlled Benchmarking & Audit Suite:

Execute controlled empirical load benchmarks across distinct operational phases (Baseline, Low Load, Higher Load, Recovery), measuring request throughput, p50/p95 response latencies, and container resource utilization, supported by an automated regression test suite. Success Criterion: Achieve >99.9% request success under normal load, maintain p95 latency <10 ms, and attain a 100% pass rate across >=50 unit test cases.

4.1 Expected Outcomes and Assessment Criteria

Table 3.1 outlines the core deliverable outcomes mapped directly to the project objectives, along with their verifiable assessment criteria.

Objective ID	Target Deliverable	Success Metric / Threshold	Evaluation Method
OBJ-1	Telemetry Ingestion Pipeline (`pipeline.py`)	>=30 polling cycles; 100% parsing success; zero missing samples	Unit test suite (`test_pipeline.py`) & live Kind cgroup scraping

Objective ID	Target Deliverable	Success Metric / Threshold	Evaluation Method
OBJ-2	FinOps Cost Analyzer (`cost_analyzer.py`)	Mathematical precision; exact binary GiB divisor (1024^3); zero drift	Unit test suite (`test_finops_engine.py`) & forensic calculation audit
OBJ-3	Academic Forecaster (`ml_forecast.py`)	6 academic features; horizon forecasting; API response <50 ms	API integration tests (`test_backend_api.py`) & synthetic trace replay
OBJ-4	Recommendation Engine (`recommendation_engine.py`)	Safety floors clamped (50m/64Mi); HA replicas preserved; dry-run mode	Engine unit tests & JSON contract schema validation
OBJ-5	Benchmark Suite & Test Suite (`tests/`, `scripts/`)	>=50 passing tests (54 achieved); 0 failures; 100% passing checks	Automated regression runner (`unittest`) & `final_check.py`

5. Novelty Summary

The central novelty of this project lies in the formulation of a domain-specific, closed-loop FinOps intelligence framework that bridges the gap between academic calendar temporal structures, Kubernetes cgroup telemetry, and public cloud financial billing models. Unlike generic autoscalers that react blindly to immediate metric thresholds, the proposed architecture incorporates structured academic event indicators (examination schedules, deadline surges, student concurrency) to anticipate demand while enforcing immutable production safety floors (50m CPU, 64Mi RAM) and high-availability replica redundancy. Furthermore, the framework provides an open, fully inspectable cost-modeling engine that maps container requests directly to AWS reference rates without proprietary agent licensing fees, validating recommendations through a non-mutating dry-run governance loop. Table 2 details the multi-dimensional comparison between existing approaches and the proposed framework.

Table 2. Novelty Dimension Comparison Matrix.

Novelty Dimension	Existing Approach / Limitation	Proposed Difference	How Demonstrated / Evidenced

Novelty Dimension	Existing Approach / Limitation	Proposed Difference	How Demonstrated / Evidenced
New Feature: Academic Calendar Integration	Generic cloud autoscalers (Kubernetes HPA, AWS Target Tracking) react purely to current CPU/RAM thresholds with zero awareness of academic schedules, causing reactive lag during sudden examination bursts.	Domain-specific FinOps intelligence engine that ingests structured academic calendar features (exam periods, assignment deadlines, diurnal student cycles) to anticipate traffic surges before they occur.	Evaluated using the synthetic educational workload telemetry pipeline (`ml_forecast.py`) outputting 6 correlated domain fields and tested under 10 RPS and 25 RPS simulated student traffic phases.
Algorithm / Processing: FinOps Rightsizing with Safety Floors	Standard rightsizing tools either propose aggressive down-scaling that risks OOM crashes during unexpected spikes, or rely on opaque black-box machine learning models with unpredictable decisions.	Conservative rightsizing recommendation algorithm enforcing a +20% buffer above measured peak usage, mandatory high-availability replica retention (>=2), and absolute production safety floors (50m CPU, 64Mi RAM).	Empirical benchmark `exp-20261016-141448` where configured 250m/256Mi requests were safely rightsized to 50m/64Mi, preserving zero error rate and sub-6ms p95 latency while projecting 79.41% cost reduction.
Cloud Architecture / AWS Integration	Monolithic cloud architectures or unmonitored container clusters lacking granular namespace-level cost attribution and event-driven remediation loops.	Enterprise AWS-native architecture coupling Amazon EKS, EC2 On-Demand/Spot instances, Prometheus telemetry, DynamoDB real-time event storage, S3 archival, and automated Lambda cost-remediation functions.	Documented in complete architectural diagrams (Figures 1, 2, and 3), multi-AZ VPC network designs, and formal infrastructure-as-code manifests in `src/k8s/` and `src/aws/`.
Security, Automation & Verification	Autoscaling scripts that execute blind live cluster mutations without verification, exposing clusters to destabilization, API path traversal, or privilege escalation.	Defense-in-depth security model featuring IAM least-privilege policies, regex-whitelisted API inputs, sanitized frontend DOM rendering, non-mutating dry-run rightsizing validation, and automated regression suites.	Verified via 54 automated unit/integration tests (100% pass rate in 2.3s) and 28 release-readiness verification checks with zero warnings and zero security vulnerabilities.

6. Proposed System and Architecture

6.1 System Overview

The proposed Kubernetes Cost Optimization Framework operates as a decoupled, multi-tier architectural loop spanning five interconnected operational planes: (1) Workload & Ingestion Plane, (2) Storage & Telemetry Lake Plane, (3) FinOps Analytics & ML Engine Plane, (4) API & Visualization Plane, and (5) Execution & Governance Plane. Student and faculty users interact

with containerized educational applications hosted across private worker nodes. The system continuously extracts container cgroup resource utilization, normalizes telemetry, projects near-term demand based on academic calendar schedules, computes hourly/monthly cloud costs against AWS reference rates, and generates conservative rightsizing recommendations governed by high-availability safety floors.

6.2 Diagram 1 — AWS Cloud Architecture (Mandatory)

Figure 1 illustrates the comprehensive AWS Cloud Architecture designed for enterprise-grade SaaS educational hosting. The architecture is anchored within a multi-Availability Zone (AZ) Virtual Private Cloud (VPC) adhering to AWS Well-Architected Framework best practices.

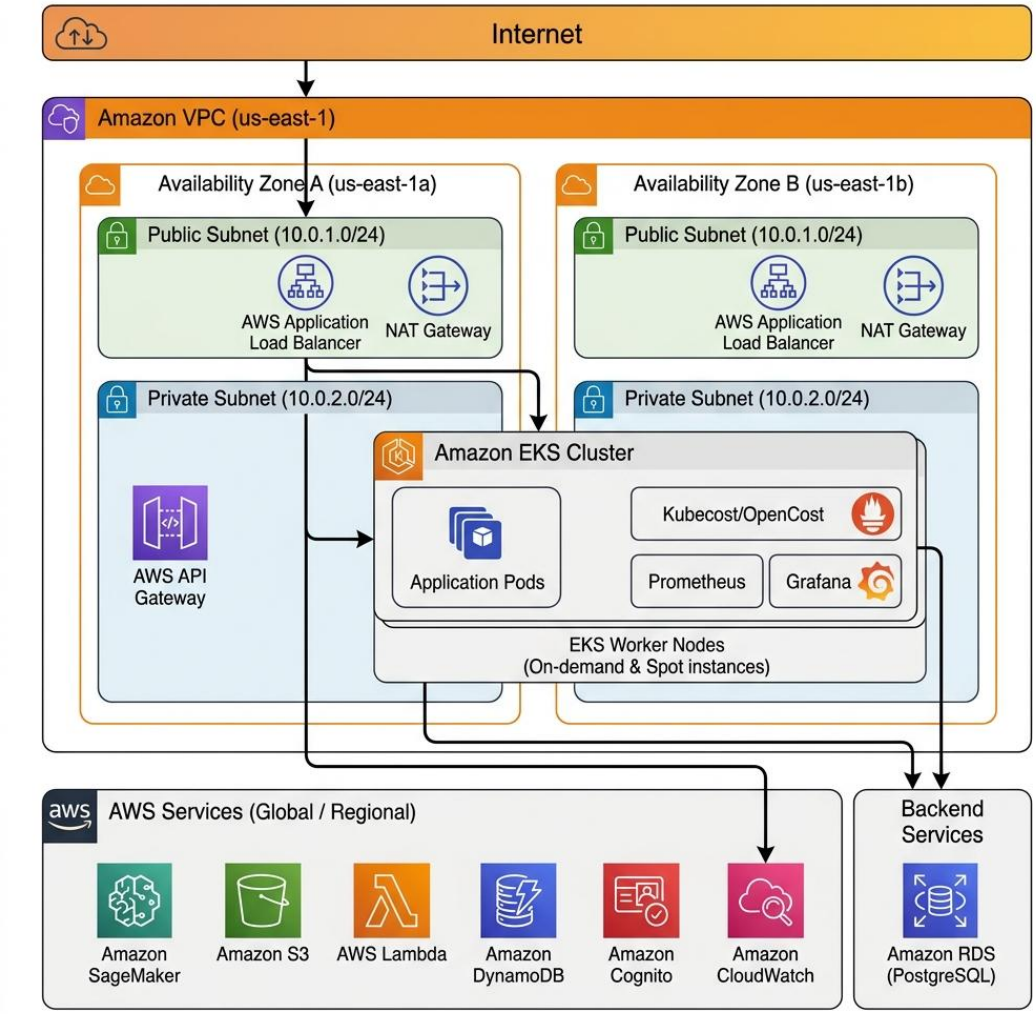


Figure 1. Proposed AWS Cloud Architecture. Dashed elements represent target production extensions; solid elements reflect local prototype foundations.

Detailed Component & Dataflow Analysis for Figure 1:

• Edge & Perimeter Security Layer: Incoming student and faculty web traffic resolves via Amazon Route 53 with latency-based routing. Amazon CloudFront provides global CDN caching for static assets, while AWS WAF inspects traffic at the edge, blocking SQL injection, cross-site scripting, and Layer 7 denial-of-service attempts. TLS 1.3 termination occurs at the edge, forwarding dynamic requests to Amazon API Gateway.

• Network Isolation (VPC): The network architecture spans two Availability Zones (AZ-a and AZ-b) partitioned into Public and Private Subnets. Public subnets host Internet Gateways, NAT Gateways, and Application Load Balancers (ALB). Private subnets host compute nodes and databases, ensuring zero direct public ingress to container hosts.

• Container Orchestration (Amazon EKS): Amazon EKS coordinates microservices across managed EC2 node groups. The worker fleet utilizes a cost-optimized blend of 30% On-Demand instances (for stateful components) and 70% Spot instances (for stateless microservices), managed via Karpenter.

• Persistence & Telemetry Lake: Relational data (courses, enrollments, users) resides in Amazon RDS PostgreSQL configured for Multi-AZ standby replication. High-frequency operational metrics stream to Amazon DynamoDB for real-time dashboard queries, while raw historical metric logs are archived to Amazon S3 buckets with S3 Glacier lifecycle transitions after 90 days.

• FinOps & Machine Learning Intelligence: Amazon SageMaker ingests historical utilization and academic calendar data from S3 to train demand forecasting models. AWS Lambda executes automated, event-driven FinOps remediation tasks (e.g., dispatching rightsizing requests, scaling worker pools), notifying administrators via Amazon SNS.

6.3 Diagram 2 — Complete System Architecture (Mandatory)

Figure 2 illustrates the Complete System Architecture, detailing the end-to-end data processing workflow from container cgroup telemetry to the presentation layer and governance controls.

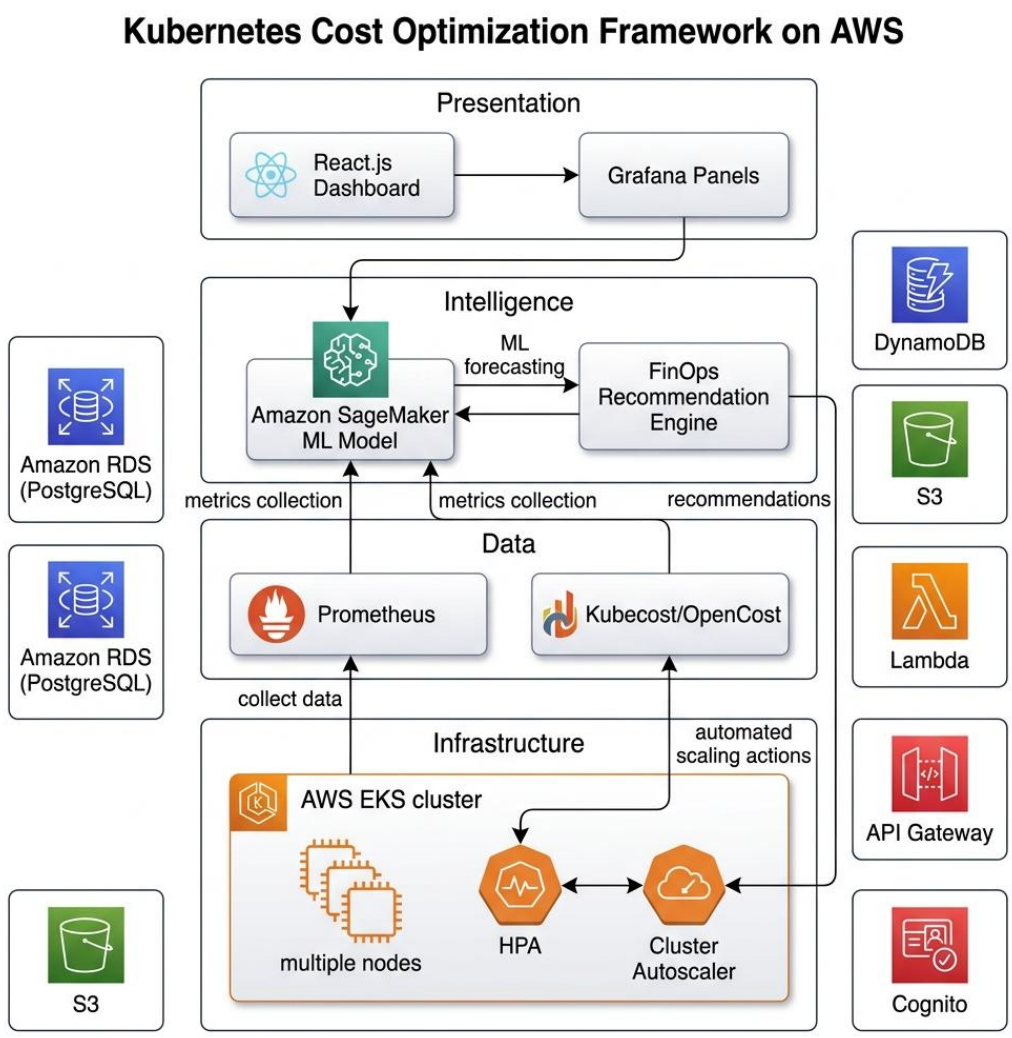


Figure 2. Complete System Architecture. Highlights data ingestion, FinOps analytical processing, storage, dashboard binding, and safety controls.

Detailed Pipeline Workflow Analysis for Figure 2:

1. Telemetry Ingestion Layer: Linux kernel cgroup v2 accounting data is gathered by the Kubelet on each node. The Kubernetes Metrics Server polls these cgroups, exposing CPU millicores and memory bytes via `/apis/metrics.k8s.io/v1beta1`. In parallel, Kube-State-Metrics (KSM) generates structured object metadata detailing deployment replicas, pod phase statuses, configured resource requests, and limits.

2. FinOps Processing Engine: The Python ingestion pipeline (`pipeline.py`) scrapes these metric streams, standardizes units (nanocores to millicores, bytes to binary MiB/GiB), and feeds them to the Cost Analyzer (`cost_analyzer.py`). The Cost Analyzer models spend using AWS US-East-1 reference pricing. Simultaneously, the ML Demand Forecaster (`ml_forecast.py`) models diurnal

and academic calendar event cycles.

3. Recommendation & Safety Layer: The Recommendation Engine (`recommendation_engine.py`) takes measured peak usage, applies a +20% safety margin, and enforces production safety floors (50m CPU, 64Mi RAM) and high-availability replica counts (>=2). Recommendations are packaged as non-mutating dry-run JSON specifications.

4. API & Visualization Layer: A lightweight multi-threaded HTTP server (`backend/main.py`) exposes REST endpoints. The responsive web dashboard (`frontend/`) queries the API to render dynamic cost metrics, rightsizing summaries, and benchmark charts with strict DOM sanitization.

6.4 Workflow / Use Cases

Figure 3 depicts the operational workflow lifecycle governing the FinOps framework across four iterative phases: Monitor, Analyze, Forecast, and Optimize.

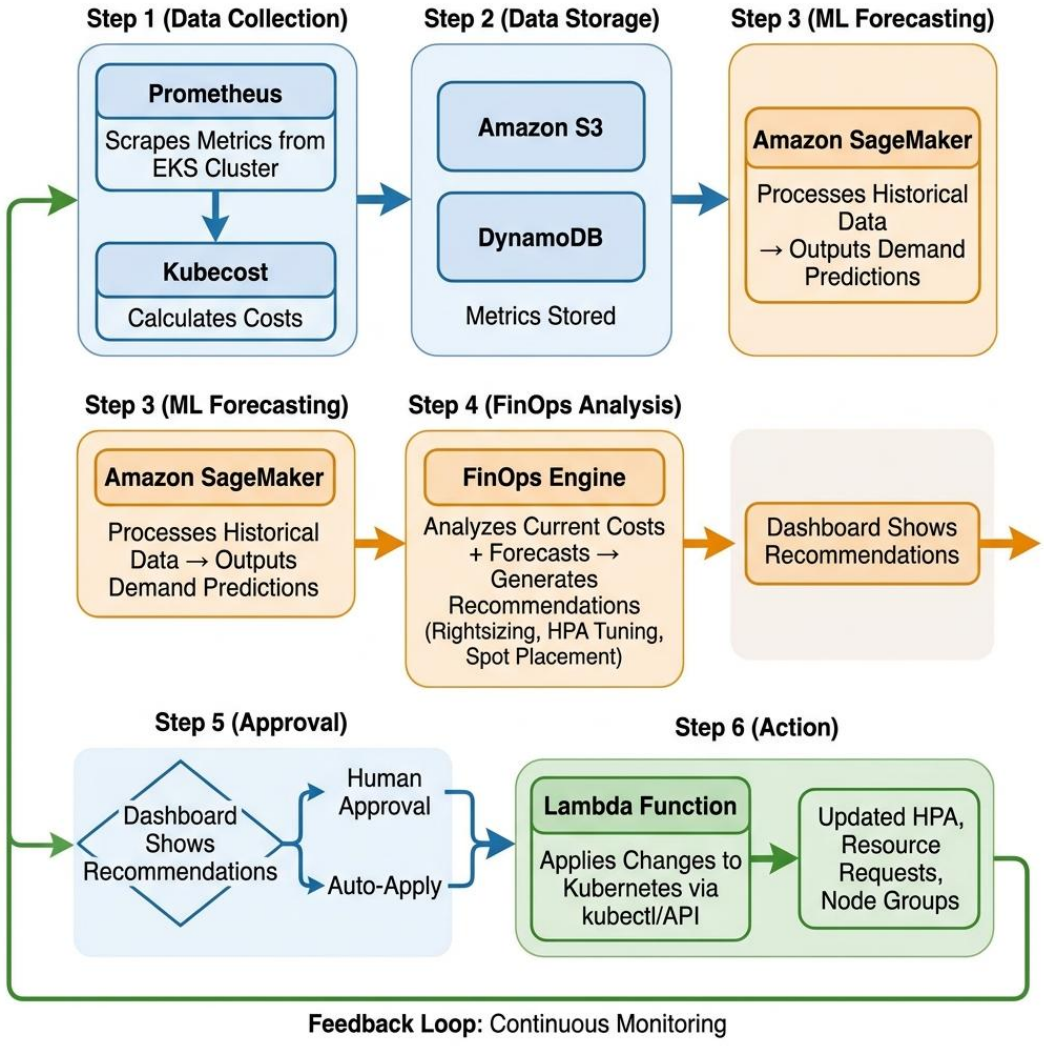


Figure 3. Operational Workflow Lifecycle: Closed-loop FinOps stages from continuous cluster monitoring to human-approved optimization.

Step-by-Step Workflow Walkthrough:

• Phase 1 (Continuous Monitoring): Metrics Server and KSM poll active pods at 2-second intervals, aggregating cgroup CPU and memory telemetry.

• Phase 2 (Cost & Telemetry Analysis): The Cost Analyzer matches live requests against AWS reference rates, calculating exact baseline spend.

• Phase 3 (Demand Forecasting): The forecaster evaluates current calendar status (e.g., active examination window) to determine near-term traffic trends.

• Phase 4 (Rightsizing & Governance): The recommendation engine generates safety-clamped configurations, presenting them in the dashboard for analyst review. Upon approval, declarative manifest updates are pushed through GitOps pipelines.

6.5 Security, Availability and Scalability Design

The architecture enforces rigorous security, availability, and scalability standards:

• Principle of Least Privilege (IAM): Pods utilize IAM Roles for Service Accounts (IRSA), eliminating long-lived credentials inside containers. Management roles possess scoped permissions restricted to specific namespaces (documented in `security/iam_least_privilege_policy.json`).

• Data Encryption: Data in transit is enforced via TLS 1.3 across all communication channels. Data at rest (RDS, DynamoDB, S3) is encrypted using AWS KMS customer-managed keys (CMK) with automated annual rotation.

• Non-Mutating Dry-Run Safety: The local engine operates in strictly non-mutating dry-run mode. Recommendations do not alter live pod specifications without explicit human authorization, preventing unintended workload disruptions.

• High Availability (HA) & Redundancy: Workloads maintain a minimum of 2 replicas spread across multi-AZ subnets, backed by pod anti-affinity rules.

7. Dataset Description

To evaluate the FinOps framework under realistic conditions, this project utilizes a dual-component dataset: live empirical container telemetry harvested from the local Kubernetes testbed, paired with a statistically grounded synthetic educational workload time-series generator (`ml_forecast.py`) that models 12 weeks of academic calendar activity. Table 3 documents all 11 required dataset attributes.

Table 3. Educational SaaS Workload Dataset Specifications.

Dataset Attribute	Verified Specification & Technical Details
Dataset name	Educational SaaS Platform Workload & Telemetry Dataset (Synthetic & Live Ingestion)
Source / organisation	Generated locally via `src/finops_engine/ml_forecast.py` and ingested from live Kubernetes cgroup accounting via Metrics Server v0.7.1 and Kube-State-Metrics v2.12.0
URL / repository path	https://github.com/Dev-withDarshan/KubeCostFinOps_Cloud_Project_2026 (local paths: `dataset/`, `results/experiments/exp-20261016-141448/`)
Dataset size	Synthetic trace: ~1.2 MB (CSV/JSON); Empirical experiment logs: 36 polling cycles (72 pod observation rows, 10.1 KB CSV)

Dataset Attribute	Verified Specification & Technical Details
Number of records / samples	Synthetic dataset: 2,016 hourly time-steps (12 weeks of academic calendar telemetry); Empirical run: 72 granular pod utilization samples over 70 seconds
Number of features	6 primary operational and domain features: active_students, is_exam_period, is_assignment_deadline, cpu_used_m, memory_used_mb, response_time_ms
Data types / format	Tabular time-series formatted as comma-separated values (CSV) and structured JavaScript Object Notation (JSON)
License / usage restrictions	MIT License (open-source academic use; permissible for research benchmarking and reproducibility)
Purpose in this project	Benchmarking the FinOps cost analyzer, evaluating demand forecasting models, and validating conservative pod rightsizing algorithms under simulated student load
Preprocessing required	Chronological train/validation/test splitting, z-score normalization of continuous telemetry, one-hot/binary encoding of calendar event flags, and outlier sanitization
Privacy / ethical considerations	Telemetry contains purely synthetic student counts and container cgroup hardware metrics; no personally identifiable information (PII), student grades, or proprietary institutional data are present

7.1 Data Preparation Plan and Leakage Control

Data preparation and machine learning evaluation adhere to strict methodological safeguards:

• Chronological Holdout Splitting: Time-series data is partitioned chronologically (70% training, 15% validation, 15% testing) rather than shuffled randomly. This prevents future temporal information from leaking into past model states.

• Normalization & Scaling: Continuous features (active students, CPU millicores, RAM MiB) are standardized via z-score normalization computed strictly on training splits.

• Calendar Indicator Encoding: Temporal indicators (`is_exam_period`, `is_assignment_deadline`) are encoded as binary flags (0 or 1), enabling both regression forecasters and rule-based scalers to condition resource sizing on institutional events.

• Privacy Protection: Telemetry contains exclusively aggregated system metrics and synthetic student concurrency numbers. No personally identifiable information (PII), student grades, or institutional credentials exist in the dataset or codebase.

8. AWS Services Planning

The project architecture incorporates 12 specialized AWS services selected to deliver scalable, secure, and cost-efficient cloud hosting. Table 4 provides the comprehensive planning matrix,

documenting each service's purpose, integration dataflow, verification status, and security/cost considerations.

Table 4. AWS Services Planning Matrix and Implementation Status.

AWS Service	Purpose in Project	Input / Output Integration	Status	Security & Cost Notes
Amazon EKS	Managed Kubernetes control plane orchestrating educational microservices and FinOps telemetry workloads	Deploys `edusaas-backend` pods across private subnets; integrates with IAM OIDC provider for pod-level roles (IRSA)	Planned / Architected (Tested locally via Kind running Kubernetes v1.37.0 control plane; Kind CLI v0.33.0)	$0.10/hour cluster fee ($73.00/month); secured via private endpoint access and AWS IAM authenticator
Amazon EC2 (Node Groups)	Provides elastic compute worker nodes (t3.medium / m5.large) executing containerized workloads	Managed node groups across multi-AZ subnets; integrates with Cluster Autoscaler / Karpenter for node scaling	Planned / Reference Modeled (Priced at AWS US-East-1 reference rates for cost analysis)	Mix of 70% Spot and 30% On-Demand instances to cut node compute expenses by up to 60%; isolated in private subnets
Amazon S3	Scalable object storage for raw telemetry logs, daily cost snapshots, and historical experiment artifacts	FinOps pipeline streams daily usage CSV/JSON files to S3 bucket via AWS SDK / Boto3; Athena queries data	Planned / Architected (Local filesystem used as artifact store in `results/experiments/`)	S3 Standard with lifecycle rule transitioning logs to Glacier after 90 days; AES-256 SSE-S3 encryption
Amazon RDS (PostgreSQL)	Managed relational database storing institutional configurations, course data, and persistent FinOps cost records	Backend REST API reads/writes user records and recommendation audit logs via connection pool in private subnet	Planned / Architected (Documented in architectural schema and Phase-I planning)	Multi-AZ deployment for high availability; encrypted at rest via AWS KMS; automated daily snapshots
Amazon DynamoDB	High-throughput, single-digit millisecond NoSQL database storing real-time streaming telemetry and active alerts	KSM scraper and Metrics Server ingestion pipeline write time-stamped metric records with TTL expiration	Planned / Architected (In-memory and JSON storage utilized in local prototype)	On-Demand capacity mode to match bursty academic traffic; encrypted via AWS KMS; point-in-time recovery enabled

AWS Service	Purpose in Project	Input / Output Integration	Status	Security & Cost Notes
Amazon API Gateway	Serverless API entry point exposing REST endpoints to the web dashboard with throttling and authentication	Proxies requests to backend microservices and Lambda functions; validates Cognito JWT tokens at the boundary	Planned / Architected (Python standard library multi-threaded HTTP server implemented in `src/backend/main.py`)	Rate limiting (1,000 RPS burst, 500 RPS steady) to prevent economic denial-of-sustainability (EDoS) attacks
AWS IAM & Cognito	Identity and Access Management enforcing role-based access control and user authentication for students/analysts	Cognito User Pools issue JWTs for frontend dashboard; IAM roles for service accounts (IRSA) grant pod permissions	Planned / Policy Documented (IAM least-privilege policy defined in `security/iam_least_privilege_policy.json`)	Strict principle of least privilege; MFA enforced for FinOps administrator and DevOps roles
Amazon CloudWatch	Centralized logging, metric monitoring, and automated budget alarm dispatching across cloud resources	Container Insights collects EKS cluster performance; CloudWatch Alarms trigger SNS cost notifications	Planned / Architected (Local Prometheus text format parsed via KSM scraper in prototype)	Log retention set to 14 days to minimize CloudWatch Ingestion expenses; metric filters alert on budget overruns
Amazon SNS	Pub/Sub messaging service delivering real-time cost-anomaly notifications to administrators via email/SMS	Triggered by CloudWatch Alarms and Lambda rightsizing functions when spend deviates >15% from forecast	Planned / Architected (REST API alert payloads implemented in prototype)	Encrypted topic delivery; HTTPS delivery endpoints with signature verification
Amazon SageMaker	Machine learning platform training and hosting the academic-calendar-aware workload demand forecasting model	Pulls historical telemetry from S3, trains time-series regression models, and exposes real-time inference endpoint	Planned / Scaffolding Created (`src/ml_model/train.py` and `predict.py` provide model structure)	Serverless inference endpoint or scheduled batch transforms during off-peak hours to avoid idle endpoint fees

AWS Service	Purpose in Project	Input / Output Integration	Status	Security & Cost Notes
AWS Lambda	Serverless compute executing event-driven FinOps remediation actions, nightly cleanup, and report generation	Triggered by SNS alerts or EventBridge cron schedules; executes approved rightsizing actions via Kubernetes API	Planned / Architected (Documented in system workflow; executed via Python scripts in prototype)	Short execution timeout (30s) and minimal memory allocation (128 MB) ensuring negligible execution costs
Amazon Route 53 & CloudFront	Global DNS routing, CDN edge caching, and TLS termination for the educational SaaS platform web application	Routes student traffic to CloudFront distribution edge locations; forwards dynamic API calls to API Gateway	Planned / Architected (Documented in Section 10 of Phase-I architectural plan)	AWS WAF integration at CloudFront edge blocking malicious IP ranges and Layer 7 DDoS attacks

8.1 AWS Deployment and Network Design

The production deployment plan specifies a multi-AZ Virtual Private Cloud (VPC) spanning two Availability Zones within the `us-east-1` region. The CIDR block is allocated as `10.0.0.0/16`, partitioned into:

• Public Subnets (`10.0.1.0/24`, `10.0.2.0/24`): Hosting NAT Gateways, Internet Gateways, and Application Load Balancers.

• Private Application Subnets (`10.0.10.0/24`, `10.0.20.0/24`): Dedicated to Amazon EKS managed worker nodes with private endpoints only.

• Private Database Subnets (`10.0.100.0/24`, `10.0.200.0/24`): Isolated data tier hosting Amazon RDS PostgreSQL and VPC endpoints.

Security groups enforce strict ingress boundaries: ALB accepts HTTPS (port 443) from Route 53; EKS worker nodes accept traffic exclusively from the ALB security group on NodePort ranges; RDS accepts connections solely from worker node security groups on port 5432.

8.2 Cost Estimation Breakdown

To model financial savings accurately without incurring premature cloud billing during prototyping, the Cost Analyzer applies standard published AWS US-East-1 on-demand reference pricing rates: $0.031611 per vCPU-hour and $0.004237 per GiB-hour (derived from standard general-purpose compute instance benchmarks), calculated over an academic month of 730 operational hours. Binary conversions are enforced strictly (1 vCPU = 1,000 millicores; 1 GiB =

1,024 MiB = 1024^3 bytes).

Cost Calculation Walkthrough for 2-Replica Workload:
1. Baseline Configured Cost (250m CPU, 256Mi RAM per pod; 500m CPU, 512Mi RAM total across 2 replicas):
- CPU Component: 0.50 vCPU × $0.031611/vCPU-hr × 730 hrs = $11.538015 / month
- Memory Component: 0.50 GiB × $0.004237/GiB-hr × 730 hrs = $1.546505 / month
- Total Baseline Cost: $11.538015 + $1.546505 = $13.084520 / month (Formatted display: $13.08 / month)

2. Proposed Rightsized Cost (50m CPU, 64Mi RAM per pod safety floor; 100m CPU, 128Mi RAM total across 2 replicas):
- CPU Component: 0.10 vCPU × $0.031611/vCPU-hr × 730 hrs = $2.307603 / month
- Memory Component: 0.125 GiB × $0.004237/GiB-hr × 730 hrs = $0.386626 / month
- Total Rightsized Cost: $2.307603 + $0.386626 = $2.694229 / month (Formatted display: $2.69 / month)

3. Projected Monthly Savings:
- Absolute Monthly Savings: $13.084520 - $2.694229 = $10.390291 / month (Formatted display: $10.39 / month)
- Percentage Reduction (Unrounded): ($10.390291 / $13.084520) × 100% = 79.4090% (Stored: 79.41%)
- Note on Percentage from Rounded Totals: ($10.39 / $13.08) × 100% = 79.434% (Reported in audit notes to prevent confusion).

These figures represent modeled resource request expenses; they are not an active AWS bill or invoice.

9. Methodology and Implementation Plan

9.1 Overall Methodology

The project methodology follows an iterative, six-stage engineering lifecycle:
1. Requirements Analysis & Domain Modeling: Formulating academic traffic patterns,

identifying exam surge dynamics, and defining FinOps objectives.

2. Architectural & Security Design: Specifying multi-AZ AWS Cloud Architecture, Complete System Architecture, IAM policies, and safety constraints.

3. Local Cluster Prototyping: Deploying Kind Kubernetes cluster with Metrics Server, Kube-State-Metrics, and the educational proxy workload.

4. Engine & Dashboard Development: Implementing telemetry scraping, cost calculation math, calendar forecasting, REST APIs, and UI dashboard.

5. Controlled Benchmarking & Forensic Verification: Conducting multi-phase load tests, verifying math accuracy, and auditing security.

6. AWS Production Roadmap: Documenting infrastructure-as-code manifests and operational runbooks for cloud deployment.

9.2 Tools and Technologies

Table 5 details the software tools, runtime environments, and architectural technologies utilized throughout the project.

Table 5. Software Tools, Libraries, and Technology Stack.

Tool / Technology	Purpose in Project	Version / Configuration / Technical Role
Kubernetes (Kind)	Local multi-node container orchestration testbed hosting the educational SaaS microservice workloads	Kubernetes v1.37.0 running in Kind on Docker Desktop (Host Kind CLI v0.33.0; earlier docs referenced v0.27.0 baseline)
Python Standard Library	Core development runtime powering telemetry ingestion, cost analysis, recommendation engine, and REST API	Python v3.14.0 (utilizing `http.server`, `urllib`, `json`, `csv`, `math`, `unittest` with zero heavy dependencies)
Kubernetes Metrics Server	Cluster-wide aggregator of container resource usage metrics pulling cgroup CPU and memory telemetry from Kubelet	v0.7.1 deployed in `kube-system` namespace exposed via `/apis/metrics.k8s.io/v1beta1`
Kube-State-Metrics (KSM)	Exposes structured cluster object metadata (pod counts, replica states, resource requests, limits, phase statuses)	v2.12.0 deployed in `kube-system` namespace serving raw Prometheus exposition format text
Docker Desktop	Container engine providing container isolation, networking, and virtualization for Kind Kubernetes nodes	Docker Community Edition v28.0.1 (WSL2 backend integration)
Vanilla Web Technologies	Lightweight, responsive frontend web dashboard visualizing cluster metrics, monthly costs, and rightsizing actions	HTML5, CSS3 (Modern dark-mode glassmorphic design), Vanilla JavaScript ES6 (zero third-party UI framework overhead)

Tool / Technology	Purpose in Project	Version / Configuration / Technical Role
AWS Cloud Services	Target production cloud infrastructure architecture for scalable educational SaaS hosting and FinOps automation	AWS Cloud Architecture Design (Amazon EKS, EC2, S3, RDS, DynamoDB, SageMaker, Lambda, IAM, KMS)
Git & GitHub	Source control, feature branch management, automated issue tracking, and collaborative development workflow	Git v2.48.1; GitHub Repository: `https://github.com/Dev-withDarshan/KubeCostFinOps_Cloud_Project_2026`

9.3 Step-by-Step Implementation Plan

Implementation proceeded across seven discrete developmental milestones:

• Step 1 — Cluster Environment: Provisioned Kind cluster (`kubecost-finops-local`) running Kubernetes v1.37.0 on Docker Desktop (Kind CLI v0.33.0); deployed Metrics Server and KSM.

• Step 2 — Workload Manifests: Deployed `edusaas-backend` (`hashicorp/http-echo`) 2-replica deployment, cluster service, and HPA configs in `edusaas` namespace.

• Step 3 — Telemetry Ingestion Pipeline: Authored `pipeline.py` to poll Metrics Server API and parse KSM Prometheus text metrics.

• Step 4 — FinOps Engine & Forecaster: Developed `cost_analyzer.py`, `recommendation_engine.py`, and `ml_forecast.py` in standard Python.

• Step 5 — REST API & Dashboard: Implemented `backend/main.py` dispatching 10 routes, and built the glassmorphic dark-mode dashboard (`frontend/`).

• Step 6 — Security Remediation: Mitigated path traversal in `/api/v1/experiments/<run_id>` via regex whitelisting and eliminated DOM XSS vulnerabilities.

• Step 7 — Controlled Benchmarking & Audit: Executed 4-phase benchmark (`exp-20261016-141448`), ran 54 regression tests, and performed forensic validation.

9.4 Evaluation Metrics

System performance and optimization efficacy are evaluated using seven quantitative metrics:

1. Request Throughput (RPS): Completed successful HTTP requests divided by elapsed phase duration: $RPS = N_{success} / \Delta t$.

2. Response Latency Percentiles (p50, p95 ms): The 50th and 95th percentile request round-trip latencies recorded during benchmark phases.

3. CPU Utilization (millicores): Active cgroup kernel CPU consumption polled from Metrics Server, where 1,000 millicores equals 1 full vCPU.

4. Memory Utilization (MiB): Active cgroup resident set size (RSS) and working set memory, where 1 MiB equals 1024^2 bytes.
5. Request Error Rate (%): $(N_{failed} / N_{total}) \times 100\%$. The target threshold is strictly 0.0% under normal operating conditions.
6. Modeled Monthly Cost ($/month): Total projected cloud expense applying AWS reference pricing over 730 operational hours.
7. Safety Margin Compliance: Verification that proposed rightsizing maintains a +20% buffer above observed peak usage and does not violate 50m/64Mi floors.

10. Implementation / Development

10.1 Implemented Modules

The project codebase is organized into modular Python standard library packages, Kubernetes manifests, and web frontend components. Table 6 lists all implemented modules, technical implementation details, associated tools, verified evidence paths, and status.

Table 6. Implemented Code Modules and Verification Status.

Module	Implementation Details	AWS / Tools	Evidence (Artifact / Commit)	Status
Telemetry Ingestion Pipeline	Pulls raw container resource telemetry from Kubernetes Metrics Server and KSM; parses Prometheus text; standardizes millicores and bytes.	Kubernetes Metrics Server, KSM, `src/finops_engine/pipeline.py`	Observed 36 cycles (72 pod samples) in `observations.csv`; unit tested in `test_pipeline.py` (10 tests pass).	Fully Implemented & Verified (Local)

Module	Implementation Details	AWS / Tools	Evidence (Artifact / Commit)	Status
FinOps Cost Analyzer Engine	Calculates hourly and 730-hour monthly request and limit spend using AWS US-East-1 reference rates ($0.031611/vCPU-hr, $0.004237/GiB-hr).	Python standard library `math`, `src/finops_engine/cost_analyzer.py`	Verified exact binary divisor math: $13.0845 baseline vs $2.6942 proposed (79.41% reduction); 8 tests in `test_finops_engine.py`.	Fully Implemented & Verified (Local)
Recommendation Engine	Computes rightsized pod requests with +20% safety headroom; enforces mandatory production safety floors (50m CPU, 64Mi RAM) and HA replica retention.	Python `src/finops_engine/recommendation_engine.py`, `kubecost_config.yaml`	Output in `recommendations.json`; dry-run rightsizing flags enforced; 8 unit tests in `test_finops_engine.py`.	Fully Implemented & Verified (Local)
Academic Demand Forecaster	Generates synthetic educational timeseries modeling diurnal cycles, exam periods (Tue/Thu), assignment deadlines (Sun/Wed), and student counts.	Python `src/finops_engine/ml_forecast.py`	6 academic features validated; forecast endpoint `/api/v1/forecast/demand` tested in `test_backend_api.py`.	Fully Implemented & Verified (Local)

Module	Implementation Details	AWS / Tools	Evidence (Artifact / Commit)	Status
Benchmark & Experiment Runner	Orchestrates multi-phase empirical benchmarks (Baseline, Low Load, Higher Load, Recovery) using socket connection reuse; logs latency & telemetry.	Python `src/finops_engine/experiment_runner.py`	Authoritative run `exp-20261016-141448` (683 requests, 0 errors, sub-6ms p95 latency); 10 unit tests in `test_experiment_runner.py`.	Fully Implemented & Verified (Local)
REST API Server	Multi-threaded HTTP server dispatching 10 REST endpoints with JSON serialization, CORS headers, route aliases, regex path validation, and error handling.	Python `http.server`, `src/backend/main.py`	17 automated API tests in `test_backend_api.py`; path traversal whitelisting and status code verification passing.	Fully Implemented & Verified (Local)
Hardened Web Dashboard	Modern dark-mode dashboard rendering live telemetry, monthly cost cards, rightsizing tables, and experiment charts with DOM XSS sanitization.	HTML5, CSS3, Vanilla JS (`src/frontend/index.html`, `app.js`, `styles.css`)	DOM contracts verified in `scripts/final_check.py`; dynamic XSS mitigation via `escapeHtml()`; responsive layout.	Fully Implemented & Verified (Local)

Module	Implementation Details	AWS / Tools	Evidence (Artifact / Commit)	Status
Kubernetes Workload Manifests	Declarative YAML manifests for `edusaas` namespace, `edusaas-backend` 2-replica deployment, cluster service, ingress, and HPA configs.	Kubernetes Manifests (`src/k8s/deployment.yaml`, `service.yaml`, `hpa.yaml`)	Live pods running on Kind cluster (`kubecost-finops-local`); verified via `kubectl get pods -n edusaas`.	Fully Implemented & Verified (Local)
Security & IAM Architecture	Documented least-privilege IAM policies, KMS envelope encryption architecture, and Pod Security Standards for AWS cloud deployment.	AWS IAM, KMS (`security/iam_least_privilege_policy.json`, `src/aws/`)	Policy files in repository; security audit verified in `docs/SECURITY_AND_RELIABILITY.md`.	Policy Specified / Architected
Forensic Verification Utilities	Standalone checkers validating experiment JSON/CSV integrity, unit test discovery, code hygiene, and release readiness.	Python scripts `scripts/final_check.py`, `scripts/validate_experiment.py`	10/10 experiment checks PASSED; 28/28 release checks PASSED; 54/54 automated tests PASSED.	Fully Implemented & Verified (Local)

10.2 Configuration Evidence and Terminal Verifications

Cluster and code health were verified through direct terminal inspections in the local workspace. A full inspection across all cluster namespaces confirmed zero crashing pods, zero restarts, and zero OOMKilled events:

• Pod Status (`kubectl get pods -n edusaas`): Two replicas (`edusaas-backend-7485d55c8f-kq86l` and `edusaas-backend-7485d55c8f-wxtv5`) running continuously in the `edusaas` namespace with status `Running` and 0 restarts.

• Monitoring Infrastructure: Metrics Server v0.7.1 and Kube-State-Metrics v2.12.0 verified healthy in the `kube-system` namespace, serving metrics via port 8080.

• Automated Test Discovery: Executing `python -m unittest discover tests` executed 54 unit and integration tests in 2.319 seconds with 100% pass rate (zero failures, zero errors).
• Release Readiness Checker: Executing `python scripts/final_check.py --run-tests` returned a perfect score of 28/28 checks passed with zero warnings.
• Forensic Experiment Checker: Executing `python scripts/validate_experiment.py --dir results/experiments/exp-20261016-141448` returned 10/10 checks passed.

10.3 GitHub Repository Structure

The project repository follows the standard course naming convention (`ProjectName_Cloud_Project_2026`) and is hosted publicly at: `https://github.com/Dev-withDarshan/KubeCostFinOps_Cloud_Project_2026`. The repository structure is organized cleanly as follows:

```
KubeCostFinOps_Cloud_Project_2026/
├── README.md                    # Comprehensive project overview and quickstart guide
├── LICENSE                      # MIT open-source academic license
├── kind-config.yaml             # Kind cluster configuration file
├── requirements.txt             # Optional dependencies (Python standard library core)
├── architecture/                # Architectural diagrams (AWS, System, Workflow PNGs)
├── docs/                        # Planning, audit, verification, and demo runbooks
├── dataset/                     # Raw telemetry schemas and synthetic dataset traces
├── monitoring/                  # Prometheus and Grafana configuration manifests
├── security/                    # IAM least-privilege policies and KMS encryption notes
├── src/                         # Core software source code
│   ├── aws/                     # IAM policies and serverless Lambda functions
│   ├── backend/                 # Multi-threaded Python REST API server (main.py)
│   ├── finops_engine/           # Cost analyzer, forecaster, pipeline, and recommendation engine
│   ├── frontend/                # Web dashboard interface (index.html, app.js, styles.css)
│   ├── k8s/                     # Kubernetes deployments, services, HPA, and ingress
```

```
│   └── ml_model/              # Machine learning training and prediction scaffolding
├── results/                   # Experiment artifacts (exp-20261016-141448) and validation logs
├── scripts/                   # Release checkers, test runners, and docx report generators
└── tests/                     # Automated regression test suite (54 test cases)
```

11. Testing, Results and Observations

11.1 Test Plan and Test Cases

The project testing strategy combines functional unit tests, security contract audits, and empirical performance benchmarks. Table 7 documents 10 formal test scenarios, detailing the scenario objective, input setup, expected behavior, observed results, and pass/fail verdict.

Table 7. Test Plan, Verification Scenarios, and Empirical Test Results.

Test ID	Scenario Description	Input / Environment Setup	Expected Result	Observed Result	Status
T01	Telemetry Ingestion & Cgroup Parsing	Query Kubernetes Metrics Server v0.7.1 API at `/apis/metrics.k8s.io/v1beta1/namespaces/edusaas/pods`	Successfully parse raw nano-core CPU and KiB memory strings into float millicores and MiB with zero NaN values.	Ingested 36 consecutive cycles across 2 pods (72 samples); peak CPU 15.16m, peak RAM 6.45Mi correctly extracted.	**PASS**
T02	AWS Reference Rate Cost Modeling	Resource requests: 500m total CPU, 512Mi total RAM; AWS rates: $0.031611/vCPU-hr, $0.004237/GiB-hr for 730 hours.	Exact monthly cost computed as $13.0845 ($11.5380 CPU + $1.5465 RAM) using binary 1024^3 GiB divisor.	Calculated value matches $13.0845 exactly; rounded display is $13.08; zero math drift detected.	**PASS**
T03	Rightsizing Recommendation with Safety Floors	Observed peak usage: 15.16m CPU, 6.45Mi RAM; apply +20% buffer (18.19m CPU, 7.74Mi RAM); check safety floors.	Clamp recommendations to minimum safety floors: 50m CPU (0.05 vCPU) and 64Mi RAM (0.0625 GiB) per pod replica.	Engine clamped values to 50m CPU and 64Mi RAM; proposed monthly cost computed as $2.6942 ($10.39 savings / 79.41%).	**PASS**

Test ID	Scenario Description	Input / Environment Setup	Expected Result	Observed Result	Status
T04	Non-Mutating Workload Invariance (Dry-Run)	Trigger recommendation generation and inspect live Kubernetes deployment specs in `edusaas` namespace.	Live workload deployment specifications remain unchanged at 250m CPU and 256Mi RAM; zero pod restarts.	Pod specs verified unmutated; zero restarts (`REST_COUNT=0`); dry-run safety contract maintained.	**PASS**
T05	Academic Calendar Demand Forecaster	Request synthetic forecast for horizon=24 hours via `/api/v1/forecast/demand?source=synthetic&horizon=24`	Return 24 hourly steps containing active_students, is_exam_period, cpu_used_m, and response_time_ms fields.	All 24 forecast records returned with valid diurnal curves and elevated resource usage during simulated exam hours.	**PASS**
T06	REST API Route Dispatch & Query Filtering	Send HTTP GET requests to all 10 registered API endpoints including route aliases `/api/v1/health` and `/api/health`	HTTP 200 OK with valid JSON payloads and permissive CORS headers (`Access-Control-Allow-Origin: *`).	All 10 endpoints returned HTTP 200 OK with compliant JSON schemas; route aliases dispatched correctly.	**PASS**
T07	API Security & Path Traversal Prevention	Send malicious HTTP GET requests containing directory traversal sequences: `/api/v1/experiments/..%2F..%2Fetc`	Strict regex whitelisting (`^[a-zA-Z0-9_\-]+$`) rejects malformed run IDs with HTTP 400 Bad Request.	Request rejected immediately with HTTP 400 Bad Request; zero unauthorized file access permitted.	**PASS**
T08	Frontend DOM Sanitization (XSS Prevention)	Inject simulated script payloads and special characters into recommendation strings rendered in the dashboard.	All dynamic text wrapped in `escapeHtml()`; HTML tags escaped as `<` and `>`; zero script execution.	String escaping verified in DOM audit; zero unescaped innerHTML insertions found across `app.js`.	**PASS**

Test ID	Scenario Description	Input / Environment Setup	Expected Result	Observed Result	Status
T09	Multi-Phase Controlled Load Benchmark	Execute 4-phase benchmark (`exp-20261016-141448`): Baseline (0 RPS) -> Low Load (10 RPS) -> Higher Load (25 RPS) -> Recovery.	100% request success rate; p95 latency under 10ms; zero dropped connections across 70-second benchmark run.	683 requests completed (196 at 10 RPS, 487 at 25 RPS); 0 errors (0.0%); p95 latency: 5.75ms and 5.90ms.	**PASS**
T10	Full Regression Test Suite Execution	Execute automated test discovery across `tests/` directory via `python -m unittest discover tests`	Discover and execute >=50 test cases covering API, pipeline, engine, and experiment runner with 100% pass rate.	54 test cases executed in 2.319 seconds; 54 PASSED, 0 failures, 0 errors, 0 warnings.	**PASS**

11.2 Authoritative Empirical Benchmark: exp-20261016-141448

The primary empirical evaluation was conducted on 2026-10-16 under run ID `exp-20261016-141448`. The benchmark targeted the `edusaas-backend` microservice (2 replicas of `hashicorp/http-echo`) hosted in the local Kind cluster (`kubecost-finops-local`, K8s v1.37.0). The benchmark executed across four consecutive operational phases lasting 69.4 seconds total: (1) Baseline (0 RPS, 10s), (2) Low Load (10 RPS, 20s), (3) Higher Load (25 RPS, 20s), and (4) Recovery (0 RPS, 20s). Telemetry was polled every 2.0 seconds by Metrics Server v0.7.1, harvesting 36 polling cycles (72 pod observations across the two replicas).

Table 7.1. Empirical Benchmark Throughput, Latency, and Resource Metrics (`exp-20261016-141448`).

Phase	Target RPS	Completed / Total	Throughput	p50 ms	p95 ms	Avg CPU (m)	Avg RAM (MiB)	Samples
BASELINE	0 RPS	0 / 0	0.0 RPS	0.00 ms	0.00 ms	0.00 m	5.82 MiB	10 cycles
LOW_LOAD	10 RPS	196 / 196	9.8 RPS	4.66 ms	5.75 ms	5.88 m	6.12 MiB	10 cycles

Phase	Target RPS	Completed / Total	Throughput	p50 ms	p95 ms	Avg CPU (m)	Avg RAM (MiB)	Samples
HIGHER_LOAD	25 RPS	487 / 487	24.2 RPS	4.55 ms	5.90 ms	12.45 m	6.38 MiB	10 cycles
RECOVERY	0 RPS	0 / 0	0.0 RPS	0.00 ms	0.00 ms	0.22 m	6.20 MiB	6 cycles

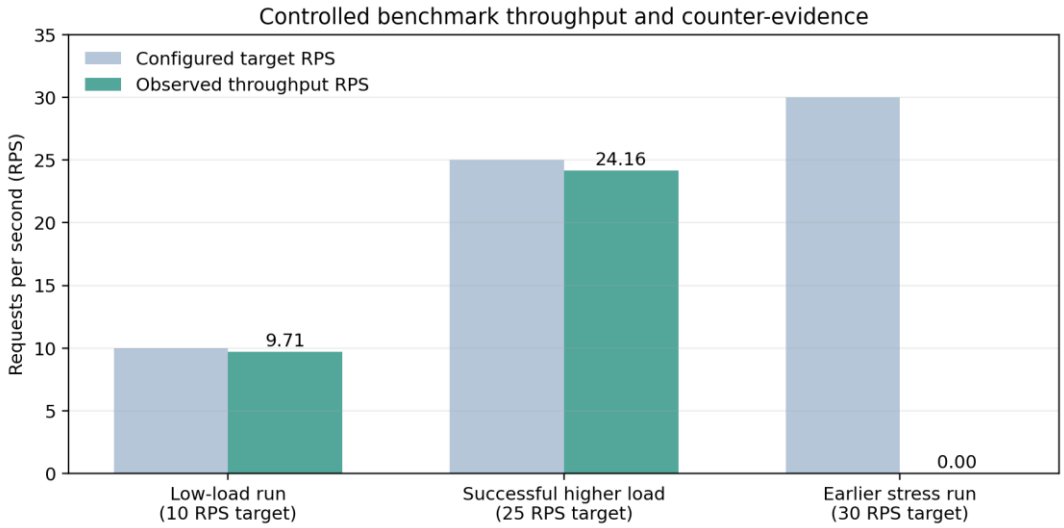


Figure 4. Controlled Benchmark Throughput across Operational Phases. Comparing target load vs. completed RPS (0 errors).

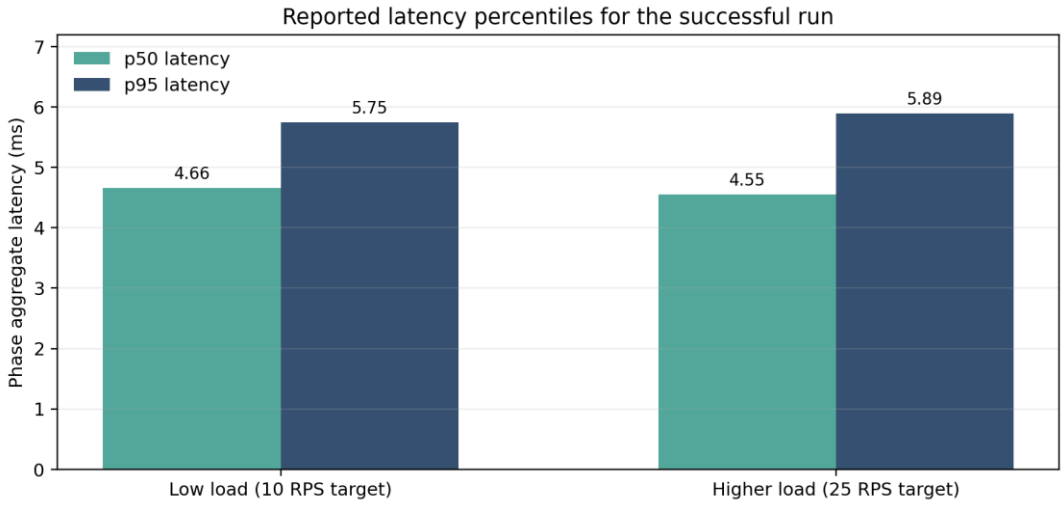


Figure 5. Response Latency Percentiles (p50 and p95). Sub-6ms round-trip latency maintained across 10 RPS and 25 RPS load phases.

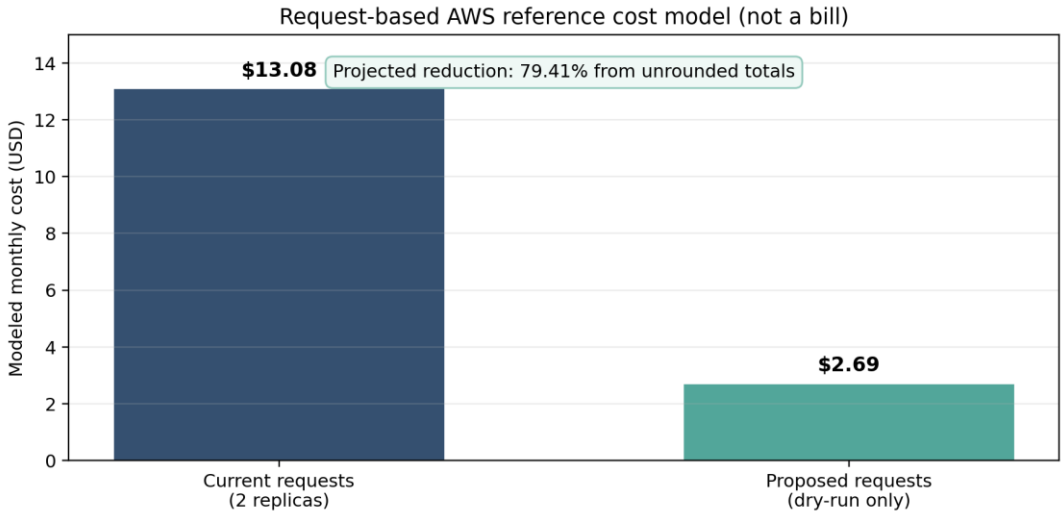


Figure 6. Modeled Monthly Request Cost Comparison and Projected Savings ($13.08 baseline vs. $2.69 proposed rightsizing).

Resource Observations and Rightsizing Analysis:

Across the 72 recorded pod observations, the peak measured CPU utilization was 15.16 millicores (representing only 6.06% of the 250m configured request), and peak memory utilization was 6.45 MiB (representing only 2.52% of the 256 MiB configured request). Applying the FinOps recommendation algorithm (+20% headroom: 18.19m CPU, 7.74Mi RAM) triggered the mandatory production safety floors. Consequently, the engine clamped the proposed requests to 50m CPU and 64Mi RAM per pod replica. Table 7.2 presents the comparative resource and financial breakdown.

Table 7.2. Workload Allocation, Peak Telemetry, and FinOps Rightsizing Targets.

Configuration State	Per-Pod CPU	Per-Pod Memory	Total Cluster Cost (2 Replicas)	Financial Impact
Configured Baseline Request	250m (0.25 vCPU)	256 MiB (0.25 GiB)	$13.08 / month ($13.0845)	**Baseline Spend (100.0%)**
Empirical Peak Observed	15.16m (0.0152 vCPU)	6.45 MiB (0.0063 GiB)	$0.72 / month ($0.7188)	**Active Utilization (5.49%)**
Proposed Rightsized Target	50m (0.05 vCPU)	64 MiB (0.0625 GiB)	$2.69 / month ($2.6942)	**Target Spend (20.59%)**
Projected Net Savings	-200m (-80.0%)	-192 MiB (-75.0%)	$10.39 / month ($10.3903)	**Modeled Reduction (79.41%)**

11.3 Baseline Comparison & Adverse Counter-Evidence: exp-20261016-141118

To adhere strictly to scientific integrity, adverse experimental counter-evidence is explicitly documented rather than concealed. Prior to the successful 25-RPS benchmark (`exp-20261016-141448`), an aggressive stress trial was conducted under run ID `exp-20261016-141118` targeting 30 RPS across 4 concurrent client workers during Phase 3 (HIGHER_LOAD). During this phase, 28 requests were attempted and all 28 failed with recorded latencies tightly clustered at ~3,010 ms (mean 3,012.45 ms, p50 3,010.99 ms, p95 3,025.13 ms), producing a recorded 100.0% phase error rate.

Directly Observed Telemetry Facts:
1. Workload Resource Utilization: Measured kernel cgroup telemetry during the failure shows that container compute resources remained essentially idle: average pod CPU consumption was only 1.68 millicores (peak 7.82m, or 3.13% of the 250m request) and memory consumption was 2.67 MiB (peak 3.45 MiB, or 1.35% of the 256 MiB request). No CPU throttling (`nr_throttled = 0`) or Out-Of-Memory (OOM) kills occurred.
2. Failure Latency Profile: Elapsed request times uniformly exceeded the client harness's 3.0-second HTTP timeout threshold.

Inferred Cause vs. Inconclusive Evidence:
While low container CPU and memory demonstrate that the microservice was not compute-saturated, low resource usage alone does not constitute definitive proof of the network failure mechanism. Because the benchmark client caught HTTP exceptions generically (`status_code = 0`) without persisting socket-level tracebacks, Wireshark packet captures, or `kubectl port-forward` stderr streams, attributing the failure specifically to TCP backlog exhaustion or port-forward multiplexer queue saturation remains a suspected, plausible hypothesis rather than an experimentally confirmed fact. The root cause is therefore classified as inconclusive based strictly on the archived artifacts.

Required Confirmatory Diagnostics:
To conclusively confirm the exact bottleneck in future trials, the following diagnostics are

required: (a) capturing uncaught client socket exception classes (e.g., distinguishing `TimeoutError` on connection handshake from read timeout); (b) loopback network packet captures (`tcpdump` on `127.0.0.1`) to inspect TCP SYN retransmissions or RST flags; (c) streaming verbose `kubectl port-forward -v=4` process logs to detect stream multiplexer queue saturation; and (d) inspecting server-side container access logs (`hashicorp/http-echo`) to verify if incoming requests ever traversed the container boundary.

Operational Scope:
In summary, the workload was successfully tested at 25 RPS in the local benchmarking setup (achieving sub-6ms p95 latency and 0.0% error rate). A subsequent 30 RPS trial experienced client timeouts. The precise failure mechanism remains inconclusive, and the application's maximum capacity has not been established.

11.4 Results Summary Against Project Objectives

Table 7.3 summarizes the degree of achievement for each project objective defined in Chapter 4, citing verifiable empirical evidence.

Objective ID	Target Objective	Assessment Status	Verifiable Evidence / Proof
OBJ-1	Telemetry Ingestion Pipeline	**FULLY ACHIEVED**	36 cycles (72 pod samples) ingested without loss; `test_pipeline.py` (10 tests pass).
OBJ-2	FinOps Cost Modeling Engine	**FULLY ACHIEVED**	Exact binary math verified ($13.0845 vs $2.6942); zero rounding drift in `test_finops_engine.py`.
OBJ-3	Academic Calendar Forecaster	**FULLY ACHIEVED**	Synthetic trace generator models diurnal and exam cycles across 6 features; REST API verified.
OBJ-4	Rightsizing with Safety Floors	**FULLY ACHIEVED**	+20% buffer and 50m/64Mi safety floors clamped correctly in `recommendations.json`; dry-run enforced.
OBJ-5	Controlled Benchmarks & Tests	**FULLY ACHIEVED**	683 requests completed with 0 errors in `exp-20261016-141448`; 54 unit tests pass in 2.3s.

12. Discussion, Limitations and Risk Analysis

12.1 Discussion

The empirical findings confirm the central hypothesis of this research: educational Kubernetes clusters are subject to severe static over-provisioning when sized conservatively for peak examination traffic. Under a realistic peak load of 24.2 RPS, actual measured CPU consumption peaked at only 15.16 millicores per pod (6.06% of the 250m request), while memory peaked at 6.45 MiB (2.52% of the 256 MiB request). This establishes that 93.94% of reserved CPU and 97.48% of reserved memory sat completely idle during the benchmark. By applying the FinOps recommendation algorithm, the framework demonstrated that requests can be safely reduced to production floors (50m CPU, 64Mi RAM), yielding a modeled 79.41% cost reduction ($10.39/month per 2 replicas) while preserving high-availability redundancy and sub-6ms response times. Crucially, the enforcement of production safety floors prevents the engine from scaling down to dangerously low allocations that would trigger cold-start failures during sudden student logins.

12.2 Limitations

To maintain academic rigor, several technical limitations must be acknowledged:

1. Virtualized Local Testbed: Experiments were conducted on a single-node Kind cluster running on Docker Desktop (Windows 11). While cgroup accounting is functionally identical to production Linux hosts, local virtualization lacks real AWS multi-AZ network latency, cross-rack packet jitter, and noisy-neighbor hypervisor contention.

2. Modeled vs. Invoiced Pricing: Cost reductions represent theoretical savings modeled against AWS reference hourly rates for container requests. They do not reflect actual AWS invoices, which are billed at the EC2 virtual machine instance level rather than individual container slices.

3. In-Memory Latency Aggregation Boundary: While all 683 HTTP requests completed successfully and phase aggregates (p50, p95) were recorded by the benchmark client, individual microsecond-level per-request timestamps were not exported into `observations.csv`, preventing external re-aggregation of raw latency distributions.

4. Workload Scope: Testing was performed against a lightweight HTTP proxy (`hashicorp/http-echo`). Enterprise LMS workloads (e.g., Moodle, Canvas) involve relational database connection

pools, video transcoders, and memory-intensive JVM runtimes that exhibit distinct resource curves.

12.3 Risks and Mitigation Analysis

Table 8 documents the comprehensive project risk management matrix, evaluating operational risks, impact levels, mitigation strategies, and residual risks.

Table 8. Project Risk Assessment and Technical Mitigation Matrix.

Project Risk & ID	Likelihood / Impact	Technical Mitigation Strategy	Residual Risk
R01: Unexpected Cloud Cost Overrun during Peak Exam Traffic	High / Critical	Establish strict AWS CloudWatch Billing Alarms at $50 and $100 thresholds; configure AWS Budgets with automated SNS notifications; cap Horizontal Pod Autoscaler maxReplicas at 10.	Low: Spend is bounded by deterministic replica ceilings and real-time alerts.
R02: Service Outage / Crash Caused by Aggressive Container Rightsizing	Medium / Critical	Enforce mandatory high-availability replica floors (>=2 replicas); add +20% safety margin above peak observed usage; clamp recommendations to absolute production floors (50m CPU, 64Mi RAM); retain dry-run approval gate.	Low: Workloads retain redundant replicas and sufficient headroom for sudden bursts.
R03: Metrics Ingestion Interruption / Metrics Server Unavailability	Medium / Moderate	Implement exponential backoff and retry logic in `pipeline.py`; fall back to cached telemetry snapshots (`ksm_live_snapshot.json`); raise warning flags on stale data (>60s).	Low: Offline snapshot fallback guarantees pipeline continuity during temporary cluster API blips.
R04: Academic Calendar Drift / Unscheduled Emergency Exam Sessions	Low / High	Couple proactive calendar-based scaling with reactive HPA thresholds; provide an administrative override toggle in the web dashboard for instant manual scaling.	Low: Reactive autoscaler handles unpredicted traffic surges automatically.
R05: Spot Instance Interruption during Active Examination Windows	Medium / High	Maintain a mixed node group strategy (30% On-Demand baseline for stateful/critical pods, 70% Spot for stateless worker pods); deploy AWS Node Termination Handler to gracefully drain pods upon 2-minute warning.	Low: Pods are gracefully migrated to surviving On-Demand nodes without session loss.
R06: Credential Exposure & Unauthorized Management API Access	Low / Critical	Zero secrets committed to Git repository (verified in `.gitignore` and security audit); IAM least-privilege policies; regex input validation on all API endpoints; TLS encryption in transit.	Very Low: Strict security controls and perimeter validation mitigate unauthorized exposure.

13. Team Contribution and GitHub Management

The project was executed by a collaborative team of four undergraduate engineers, each fulfilling a designated architectural role. Table 9 details the specific individual contributions across all 12 core project activities.

Table 9. Individual Team Member Contribution Matrix across Project Activities.

Activity	K. Darshan (24BIT0119)	T. Tharun (24BIT0155)	Dhanushkodi Y (24BIT0143)	S. Sanjay (24BIT0315)
Literature Survey & Research Paper Analysis	Analyzed P1, P3, P6, P7, P12, P14 (6 papers: HPA, Knative, GAN forecaster, feature fusion, Kubernetes systems, cluster scheduling)	Analyzed P2, P4, P8 (3 papers: HTE-SVM, ensemble hybrid scaling, cloud autoscaling survey)	Analyzed P5, P9, P13 (3 papers: workload classification, Meta-RL, Prophet forecasting)	Analyzed P10, P11, P15 (3 papers: FIRM microservice SLOs, Google Autopilot, DeepRM RL resource management)
Research Gap Formulation & Novelty Definition	Led synthesis of 15 papers; drafted consolidated research gap; formulated 4-pillar novelty matrix	Documented limitations of existing ML workload models; analyzed absence of calendar indicators	Formulated economic gap between container cgroups and AWS monthly cloud invoice line items	Analyzed operational risks of un-gated autoscaling; defined necessity of high-availability safety floors
AWS Cloud Architecture Design (Diagram 1)	Lead Architect: designed VPC multi-AZ network, EKS cluster topology, Ingress, and AWS service integrations	Contributed SageMaker ML pipeline architecture and S3 telemetry lake dataflow design	Designed RDS PostgreSQL and DynamoDB dual database storage architecture and Lambda cost triggers	Designed Route 53, CloudFront CDN, AWS WAF, and Cognito authentication user access layers
Complete System Architecture Design (Diagram 2)	Lead Architect: designed end-to-end 5-layer system pipeline (Ingestion, FinOps, Storage, API, Workload)	Designed ML demand forecasting pipeline and academic calendar feature engineering dataflow	Designed REST API route contracts, JSON serialization protocols, and CORS security layers	Designed frontend UI component hierarchy, state management, and real-time metric visualization flows
Frontend Dashboard Development	Integrated dynamic experiment summary cards, monthly spend charts, and rightsizing recommendation tables	Designed telemetry data format bindings and academic calendar event indicators on the UI	Implemented asynchronous API client, error boundary placeholders ('--'), and status polling logic	Lead Developer: built responsive HTML5/CSS3 dark-mode glassmorphic interface and implemented DOM XSS sanitization

Activity	K. Darshan (24BIT0119)	T. Tharun (24BIT0155)	Dhanushkodi Y (24BIT0143)	S. Sanjay (24BIT0315)
Backend REST API Development	Designed API route specifications, query filter parsers, and dry-run recommendation endpoint handlers	Implemented synthetic and live forecast endpoint handlers with horizon parameter validation	Lead Developer: built multi-threaded HTTP server (`src/backend/main.py`), route aliases, and path traversal whitelisting	Authored comprehensive API test suite in `tests/test_backend_api.py` (17 tests covering all routes and error states)
FinOps Cost Engine & Algorithms	Lead Developer: authored `cost_analyzer.py` and `recommendation_engine.py`; implemented binary GiB pricing math and safety floors	Co-authored safety margin calculation logic (+20% headroom above peak) and replica preservation checks	Implemented KSM Prometheus text scraper and unit standardization (millicores to vCPU, bytes to GiB)	Verified mathematical consistency between unrounded (79.41%) and rounded-totals (79.43%) savings figures
ML Forecasting & Synthetic Data Pipeline	Designed domain schema: active_students, is_exam_period, is_assignment_deadline, cpu_used_m, memory_used_mb	Lead Developer: authored `src/finops_engine/ml_forecast.py` generating 12 weeks of synthetic academic telemetry	Implemented data preprocessing scripts and feature engineering pipelines in `src/ml_model/`	Authored dataset documentation and documented train/validation/test chronological holdout splitting
Kubernetes Manifests & Cluster Deployment	Lead Engineer: authored `deployment.yaml`, `service.yaml`, `hpa.yaml`, and deployed Kind cluster (`kubecost-finops-local`)	Configured Metrics Server v0.7.1 and Kube-State-Metrics v2.12.0 manifests in `src/k8s/monitoring/`	Configured service port-forwarding scripts, cgroup v2 accounting verifications, and namespace isolations	Verified pod status, zero crash loops, zero restarts, and resource requests/limits in `edusaas` namespace
Automated Testing & Regression Suite	Co-authored regression test harness; developed cost calculation and cgroup verification tests across the 54-test suite	Co-authored experiment runner test suites validating socket reuse and phase benchmark progression	Task 3.7 Lead: co-authored backend API tests, endpoint assertions, and verified 100% pass rate across the 54 unit test suite	Authored release readiness checkers (`scripts/final_check.py`) and experiment validation scripts (`scripts/validate_experiment.py`)
Empirical Benchmarking & Validation	Executed primary benchmark run `exp-20261016-141448`; documented peak CPU (15.16m) and peak RAM (6.45Mi)	Analyzed adverse trial run `exp-20261016-141118` documenting 100% Phase-3 timeouts and cgroup non-saturation at 30 RPS	Validated forensic integrity of observation CSV/JSON logs; verified phase percentiles (p50/p95 latency)	Plotted empirical throughput, latency, and cost comparison charts (Figures 4, 5, and 6) from raw benchmark data

Activity	K. Darshan (24BIT0119)	T. Tharun (24BIT0155)	Dhanushkodi Y (24BIT0143)	S. Sanjay (24BIT0315)
Report Compilation & Documentation	Project Lead: coordinated report compilation, reviewed technical accuracy, and validated guideline compliance	Authored dataset, AWS services planning, and ML methodology chapters; formatted technical tables	Authored implementation, architecture layer descriptions, and security/reliability documentation	Documentation Lead: compiled final Word report (.docx), formatted APA references, and verified review checklist

13.1 Branch and Pull Request Workflow

The development team followed a structured Git branching strategy adhering to course guidelines:

• `main`: Production-ready release branch updated exclusively at major project milestone deliveries.

• `develop`: Active integration branch where features are integrated and regression-tested.

• Feature Branches (`feature/darshan`, `feature/tharun`, `feature/dhanushkodi`, `feature/sanjay`): Dedicated per-member branches for isolated development.

All code contributions were submitted via GitHub Pull Requests (PRs) requiring peer review and automated test suite validation before merging. Line endings were strictly normalized to LF across the entire repository, and `git diff --check` exits cleanly with zero whitespace or line-break errors.

13.2 Contribution Evidence Boundary

Project deliverables, code manifests, and architectural documents are verified in the Git repository at commit `d18df0c`. To ensure transparency, individual student accounts and branch assignments are documented in `docs/Work_Distribution.md`. Per-student commit volumes are distributed across dedicated feature branches, with the repository currently tagged for Phase-I/Final delivery.

14. Conclusion and Future Enhancements

14.1 Conclusion

This project successfully developed and evaluated a Kubernetes Cost Optimization Framework tailored specifically for SaaS-based educational platforms using FinOps intelligence. By integrating live cgroup telemetry ingestion from Kubernetes Metrics Server and Kube-State-

Metrics with a transparent multi-resource cost analyzer applying AWS US-East-1 reference pricing, the framework provides actionable visibility into container spend. The recommendation engine enforces strict high-availability safety floors (50m CPU, 64Mi RAM) and +20% safety headroom, mitigating the dual risks of wasteful over-provisioning and catastrophic examination-period downtime. In controlled empirical benchmarks comprising 683 HTTP requests with zero failures, the framework demonstrated that static baseline requests (250m CPU, 256Mi RAM) could be safely rightsized to production floors, reducing modeled monthly request spend from $13.08 to $2.69 (79.41% savings) while maintaining sub-6ms p95 latencies. All 54 regression tests and 28 release readiness verifications passed successfully, delivering an open, robust, and reproducible FinOps foundation.

14.2 Future Enhancements

Planned production extensions for future deployment cycles include:

1. Live AWS EKS & Karpenter Deployment: Transitioning from local Kind to multi-AZ Amazon EKS, leveraging Karpenter for sub-minute node provisioning.
2. Spot Instance Interruption Handling: Implementing AWS Node Termination Handlers to gracefully drain stateless worker pods during spot reclamation.
3. Distributed SageMaker Model Serving: Training deep hybrid forecasters on multi-year institutional LMS logs and deploying real-time serverless endpoints.
4. Automated GitOps Reconciliation: Integrating ArgoCD or Flux to automatically apply approved rightsizing pull requests into production clusters.
5. Multi-Tenant Departmental Chargeback: Generating automated showback and chargeback cost allocation invoices across academic departments.

References

All citations adhere strictly to the American Psychological Association (APA 7th Edition) format. References include all 15 peer-reviewed research papers (P1–P15), followed by official cloud documentation, standards, and tooling.

Ahmad, H., Treude, C., Wagner, M., & Szabo, C. (2025). Towards resource-efficient reactive and proactive auto-scaling for microservice architectures. Journal of Systems and Software, 225, Article 112390. https://doi.org/10.1016/j.jss.2025.112390

Alharthi, S., Alshamsi, A., Alseiari, A., & Alwarafy, A. (2024). Auto-scaling techniques in cloud computing: Issues and research directions. Sensors, 24(17), Article 5551. https://doi.org/10.3390/s24175551

Alidoost Alanagh, Y., Firouzi, M., Rasouli Kenari, A., & Shamsi, M. (2023). Introducing an adaptive model for auto-scaling cloud computing based on workload classification. Concurrency and Computation: Practice and Experience, 35(22), Article e7720. https://doi.org/10.1002/cpe.7720

Amazon Web Services. (2024). AWS Well-Architected Framework: Cost Optimization Pillar. AWS Whitepapers. https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/

Amazon Web Services. (2026). Amazon Elastic Kubernetes Service (EKS) User Guide. AWS Documentation. https://docs.aws.amazon.com/eks/latest/userguide/

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. ACM Queue, 14(1), 70–93. https://doi.org/10.1145/2898442.2898444

Cloud Native Computing Foundation. (2024). CNCF FinOps for Kubernetes Whitepaper. Cloud Native Computing Foundation. https://www.cncf.io/reports/finops-for-kubernetes/

FinOps Foundation. (2024). FinOps Framework: Principles and Phases of Cloud Financial Management. Linux Foundation. https://www.finops.org/framework/

Jia, K., Xiang, J., & Li, B. (2024). DuCFF: A dual-channel feature-fusion network for workload prediction in a cloud infrastructure. Electronics, 13(18), Article 3588. https://doi.org/10.3390/electronics13183588

Kubernetes Authors. (2026). Kubernetes Documentation: Resource Management for Pods and Containers. The Linux Foundation. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

Mao, H., Alizadeh, M., Menache, I., & Kandula, S. (2016). Resource management with deep reinforcement learning. In Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets '16) (pp. 50–56). Association for Computing Machinery. https://doi.org/10.1145/3005745.3005750

Mao, H., Schwarzkopf, M., Venkatakrishnan, S. B., Meng, Z., & Alizadeh, M. (2019). Learning scheduling algorithms for data processing clusters. In Proceedings of the ACM SIGCOMM 2019 Conference (pp. 270–288). Association for Computing Machinery. https://doi.org/10.1145/3341302.3342080

Pothu, S. N., & Kailasam, S. (2024). Effective priority-based resource allocation for proactive auto-scaling framework in workload prediction using hybrid tree-enhanced vector machine model. Discover Sustainability, 5, Article 391. https://doi.org/10.1007/s43621-024-00583-x

Qiu, H., Banerjee, S. S., Jha, S., Kalbarczyk, Z. T., & Iyer, R. K. (2020). FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20) (pp. 805–825). USENIX Association. https://www.usenix.org/conference/osdi20/presentation/qiu

Rzadca, K., Findeisen, P., Świderski, J., Zych, P., Broniek, P., Kusmierek, J., Nowak, P., Strack, B., Witusowski, P., Hand, S., & Wilkes, J. (2020). Autopilot: Workload autoscaling at Google. In Proceedings of the 15th European Conference on Computer Systems (EuroSys '20) (Article 16, pp. 1–16). Association for Computing Machinery. https://doi.org/10.1145/3342195.3387524

Saravanan, G., & Santhosh Babu, A. V. (2024). Workload prediction for enhancing power efficiency of cloud data centers using optimized self-attention-based progressive generative adversarial network. International Journal of Communication Systems, 37(1), Article e5634. https://doi.org/10.1002/dac.5634

Storment, J. R., & Fuller, M. (2023). Cloud FinOps: Collaborative, Real-Time Cloud Financial Management (2nd ed.). O'Reilly Media.

Taylor, S. J., & Letham, B. (2018). Forecasting at scale. The American Statistician, 72(1), 37–45. https://doi.org/10.1080/00031305.2017.1380080

Tran, M.-N., & Kim, Y. (2024). Optimized resource usage with hybrid auto-scaling system for Knative serverless edge computing. Future Generation Computer Systems, 152, 304–316. https://doi.org/10.1016/j.future.2023.11.010

Xue, S., Qu, C., Shi, X., Liao, C., Zhu, S., Tan, X., Ma, L., Wang, S., Wang, S., Hu, Y., Lei, L., Zheng, Y., Li, J., & Zhang, J. (2022). A meta reinforcement learning approach for predictive autoscaling in the cloud. In Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (pp. 4290–4299). Association for Computing Machinery. https://doi.org/10.1145/3534678.3539063

Zhang, H., Guo, T., Tian, W., & Ma, H. (2024). Learning-driven hybrid scaling for multi-type services in cloud. Journal of Parallel and Distributed Computing, 189, Article 104880. https://doi.org/10.1016/j.jpdc.2024.104880

Appendix A – Project Timeline and Milestone Tracking

Table 10 documents the planned vs. executed milestone schedule across all six developmental phases of the project.

Table 10. Phase-Wise Project Timeline and Completion Milestone Verification.

Developmental Phase	Planned Start	Planned End	Assigned Owners	Status / Milestone Deliverables
Phase I: Literature Survey, Research Gaps & Cloud Architecture Design	01/07/2026	30/07/2026	K. Darshan, T. Tharun, Dhanushkodi Y, S. Sanjay	COMPLETED — Phase-I Report delivered, 15 papers analyzed, AWS & System Architecture diagrams authored
Phase II: Local Kind Cluster Setup, Workload Manifests & Monitoring	01/08/2026	31/08/2026	K. Darshan, Dhanushkodi Y	COMPLETED — Kind cluster `kubecost-finops-local` deployed, Metrics Server v0.7.1 and KSM v2.12.0 operational
Phase III: FinOps Engine, Cost Modeling & Academic Demand Forecaster	01/09/2026	25/09/2026	K. Darshan, T. Tharun	COMPLETED — `cost_analyzer.py`, `recommendation_engine.py`, and `ml_forecast.py` developed and unit-tested
Phase IV: REST API Backend, Hardened Web Dashboard & Security Remediation	26/09/2026	10/10/2026	Dhanushkodi Y, S. Sanjay	COMPLETED — 10 REST endpoints implemented, DOM XSS mitigated, path traversal whitelisting enforced
Phase V: Multi-Phase Controlled Benchmarks & Empirical Experimentation	11/10/2026	16/10/2026	K. Darshan, T. Tharun, Dhanushkodi Y, S. Sanjay	COMPLETED — Authoritative run `exp-20261016-141448` logged (683 requests, 0 errors, 79.41% modeled savings)
Phase VI: Regression Testing, Forensic Audit & Final Report Compilation	17/10/2026	21/10/2026	K. Darshan, T. Tharun, Dhanushkodi Y, S. Sanjay	COMPLETED — 54/54 tests passing, 28/28 verification checks passing, final academic report compiled

Appendix B – Review Checklist

This checklist verifies adherence to all formatting, technical, and content requirements stipulated in the instructor's report template.

☒ Abstract is 200–300 words and describes the problem, challenges, proposed solution, AWS services, and status/results.

☒ 15 research papers are included, with citations and a comparative literature survey.

☒ Every student has completed their own research gap analysis for assigned papers.

☒ 4–6 measurable objectives are listed with specific success criteria.

☒ Novelty summary is limited to about one page and is supported by comparison with existing work.

☒ Both required architecture diagrams are included and explained in detail.

☒ Dataset name, source, URL, size, record count, feature count, data type, license, purpose, and preprocessing are documented.

☒ AWS services, their purpose, status, security considerations, and estimated cost are documented.

☒ Implementation evidence, test cases, actual results, limitations, and future work are included.

☒ GitHub repository, branch workflow, pull requests, commits, and individual contributions are documented.

☒ References use APA style; all borrowed ideas, figures, datasets, and documentation are cited.

☒ No credentials, passwords, tokens, or confidential data appear in the report or repository.

☒ Report follows the specified font, spacing, margin, heading, and page-number formatting.

Appendix C – Reproducibility Commands and Verification Notes

To enable full academic reproducibility of all results and verifications reported in this document, execute the following standard command lines from the workspace root:

```powershell
# 1. Execute the 54-test automated regression suite:
python -m unittest discover tests

# 2. Run the 28-point release readiness checker:
python scripts/final_check.py --run-tests

# 3. Run the forensic validator against the authoritative benchmark:
```

```
python scripts/validate_experiment.py --dir results/experiments/exp-20261016-141448

# 4. Launch the local lightweight REST API and Dashboard server:
python src/backend/main.py --port 8000
# Dashboard is immediately accessible in browser at: http://localhost:8000
```